

ADVANCED SCHOOL OF INFORMATION TECHNOLOGY

Department of Network Security & Computer Forensics

ANDROID PENETRATION TESTING PROJECT

A Project Report Submitted in Partial Fulfilment of the Requirements for the Programme in Security Administration and Computer Forensics

PREPARED BY	**Thando Sibakhele Msibi**
PROGRAMME	Security Administration and Computer Forensics
INSTITUTION	Advanced School of Information Technology (ASIT)
Student ID	Student1619685
Date	19/052026
email	thandomsibi@gmail.com

TABLE OF CONTENT

1 **. DECLARATION**…… .4

2 **. ACKNOWLEGEMENT**……... 5

3 **. INTRODUCTION**……. 6

4 **. OBJECTIVES** ……7

5 **. TOOLS AND ENVIROMENT SETUP** ……8

6 **. STATIC ANALYSIS**

6.1 APK Acquisition and extraction……..9

7**. ADB COMMAND AND DEVCE INTERACTION**

7.1 Installing ABD……12

8 . ADB COMMANDS……..15

8.1 adb devices and get-serialno……15

8.2 adb install ……15

8.3 Adb pull and adb pull……...15

8.4 Adb get state……16

8.5 Adb backup……...16

8.6 Adb shell dump sys……..…….16

8.7 Adb dump sys activity……..…….18

8.8 Adb shell wm…….18

8.9 Adb shell pm list packages……..……..19

8.10 Adb shell pm path <apk>…….20

9. **ADVANCE DECOMPILATION TECHNIQUES**

9.1 Downloading dex2jar and JD-GUI……23

9.2 Decompiling the app using Dex2jar…….…….24

9.3 Using JD-GUI……25

9.4 Installing and using jadx…… ..26

10. DYNAMIC ANALYSIS WITH DROZER

10.1 Installing drozer ……. 27

10.2 Retrieving package information……...…….……32

10.3 Identifying application attack surface……..…...32
10.4 Exploiting activities……..…….33
10.5 Exploiting providers info……..…….33
11**. DYNAMIC ANALYSIS WITH BURPSUIT**
11.1 Proxy interception setup…….34
11.2 Manipulating traffic……44
12. **KEY FINDINGS AND ANALYSIS**…….……..46
13**. CONCLUSION**……...……..47
14**. REFERENCES**……50

1. DECLARATION

I, Thando Msibi, hereby declare that this project report titled ***Android Pentesting Project*** is my own original work and has been carried out under the supervision of my lecturer at the Advanced School of Information Technology (ASIT). This report has not been submitted previously for any academic award or qualification at this or any other institution.

All sources of information and references that I used in this project have been duly acknowledged and cited in accordance with the accepted academic referencing conventions. Any content derived from external sources, including tools, methodologies, and technical documentation, has been appropriately credited.

I further declare that this work is a true reflection of my understanding, skills, and knowledge acquired during the course of study in the Security Administration and Computer Forensics programme. I acknowledge that any act of academic dishonesty, including plagiarism, fabrication of data, or misrepresentation, may result in disciplinary action in accordance with the institution's academic integrity policies.

Student Name: Thando Msibi	Student Signature
Student ID	Date
Lecturer Signature	Date

2. ACKNOWLEDGEMENT

First and foremost, I give glory to God for the strength, wisdom, and guidance throughout the duration of this project. This work would not have been possible without His grace.

I would like to express my sincere and heartfelt gratitude to my lecturer for their invaluable guidance, patience, and constructive feedback throughout the planning, design, and implementation of this project. Their expertise in network security and computer forensics greatly shaped the direction and quality of this work.

I also extend my appreciation to the management and staff of RSTP(Asit) for providing a conducive learning environment and access to the resources needed to complete this project. The institution's commitment to practical, hands-on training in information technology has been instrumental in my academic growth.

Special thanks go to my fellow students and classmates in the Security Administration and Computer Forensics programme for their support, encouragement, and collaborative spirit. The discussions and shared knowledge we exchanged during this journey were truly beneficial.

Name: _____

Student ID: _____

Student Signature: _____ Date: _____

Lecture's Signature: _____ Date _____

3. INTRODUCTION

In the contemporary digital era, mobile technology has fundamentally transformed the way individuals communicate, conduct business, manage finances, and access information. Among the various mobile operating systems available today, **Android** stands as the undisputed global leader, powering billions of smartphones, tablets, wearable devices, and Internet of Things (IoT) gadgets across virtually every continent. Its open-source nature, extensive customisability, and vast ecosystem of applications available through the Google Play Store and third-party distribution channels have contributed enormously to its widespread adoption. However, this very openness and ubiquity also render Android a particularly attractive target for malicious actors, cybercriminals, and advanced persistent threat groups seeking to exploit vulnerabilities for financial gain, data theft, and large-scale surveillance.

Against this , **penetration testing** has emerged as an indispensable discipline within the broad field of cybersecurity. Often referred to as ethical hacking, penetration testing involves the systematic simulation of real-world attacks against applications, systems, or networks with the explicit objective of identifying security weaknesses before malicious adversaries can exploit them. In the context of Android applications, penetration testing is uniquely challenging due to the platform's layered architecture, the diversity of hardware and software configurations across device manufacturers, the complexities of inter-process communication (IPC), and the intricate interplay between client-side application logic and server-side APIs. A thorough Android penetration test demands not only technical proficiency but also a methodical approach that integrates multiple analytical perspectives and specialised toolsets.

So in this project I structured it to two complementary analytical paradigms: **static analysis** and **dynamic analysis**. Static analysis is the meticulous examination of an application's codebase, manifest files, resources, and compiled binaries without executing the application itself. Through techniques , which I will cover deeply in the project, such as APK decompilation, disassembly, and source-code review, static analysis can reveal hardcoded credentials, insecure permission declarations, exported components, vulnerable cryptographic routines, and suspicious data flows. Conversely, dynamic analysis involves observing and interacting with the application during runtime, either on a physical device or an emulator , however I will be using an emulator, to uncover behaviour that is not apparent through static inspection alone. My aim is to identifying insecure network communications, testing for injection vulnerabilities, manipulating application state, and assessing how the application handles untrusted input under realistic operating conditions.

The practical demonstrations throughout this report are conducted using intentionally vulnerable Android applications, most notably the **InsecureBankv2** application, which is widely recognised within the cybersecurity community as a pedagogically rich target for mobile security education. I will be perfoming all testing within controlled, isolated environments using emulated and rooted Android devices, ensuring full compliance with ethical standards and legal boundaries. Every technique is documented with detailed command-line inputs, screenshots, and explanatory annotations to ensure that the procedures are transparent, reproducible, and accessible to both academic readers and aspiring security professionals.

4, Objectives

- To acquire, extract, and decompile APK files for static analysis.
- To demonstrate ADB usage for device management and data extraction.
- To perform advanced decompilation and source-code inspection.
- To identify attack surfaces using the Drozer framework on an intentionally vulnerable application.
- To configure dynamic analysis using Burp Suite for traffic interception.
- To document findings, analyse risks, and recommend mitigation strategies.

5. Tools and Environment Setup

The following open-source and industry-standard tools were used:

- **APKTool** – for decompiling APKs into readable Smali and manifest files.
- **Android Debug Bridge (ADB)** – for device communication.
- **Dex2jar + JD-GUI** – for converting DEX to JAR and viewing Java source code.
- **JADX** – for direct APK decompilation and Java/Kotlin source viewing.
- **Drozer** – for runtime security assessment of apps and device IPC.
- **Burp Suite** – for dynamic proxy interception and certificate pinning bypass.
- **Genymotion Emulator/ Android Studio emulator** and rooted Android device – for safe testing.

All tools were installed on a Windows environment, with necessary paths added to the system environment variables.

6. STATIC ANALYSIS

6.1 APK Acquisition and extraction.

As I have mentioned that static analysis involves viewing and examining the app without execution. So I need to download any Apk file form trusted sources as part of acquisition goal analysis .

Downloading any APK file:
Downloading any APK file for testing , below is a screenshot for the APK file downloaded. It is an InsecureBanking App (InsecureBankingv2.apk) which is found on Github. It is vulnerable application for penetration testing . It is successfully saved in the working directory. It confirm the file is ready for static analysis and shows basic file properties such as file and modification date.

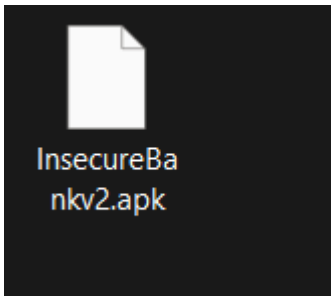


Figure 1: Screenshot of the downloaded APK file

Next step is zipping the file and unzipping the file to show all its content or file directories, below are screenshots of the zipped and extracted file and its directories. I need to rename it to .zip format (do not compress it , just rename it to .zip) , This will change the file to a zip archive then RIGHT ClICK the ZIP archive and extract the file to reveal its internal structure (classes.dex , resources.arsc, AndroidManifest.xml, etc). Figure 2 shows the file content after extraction.

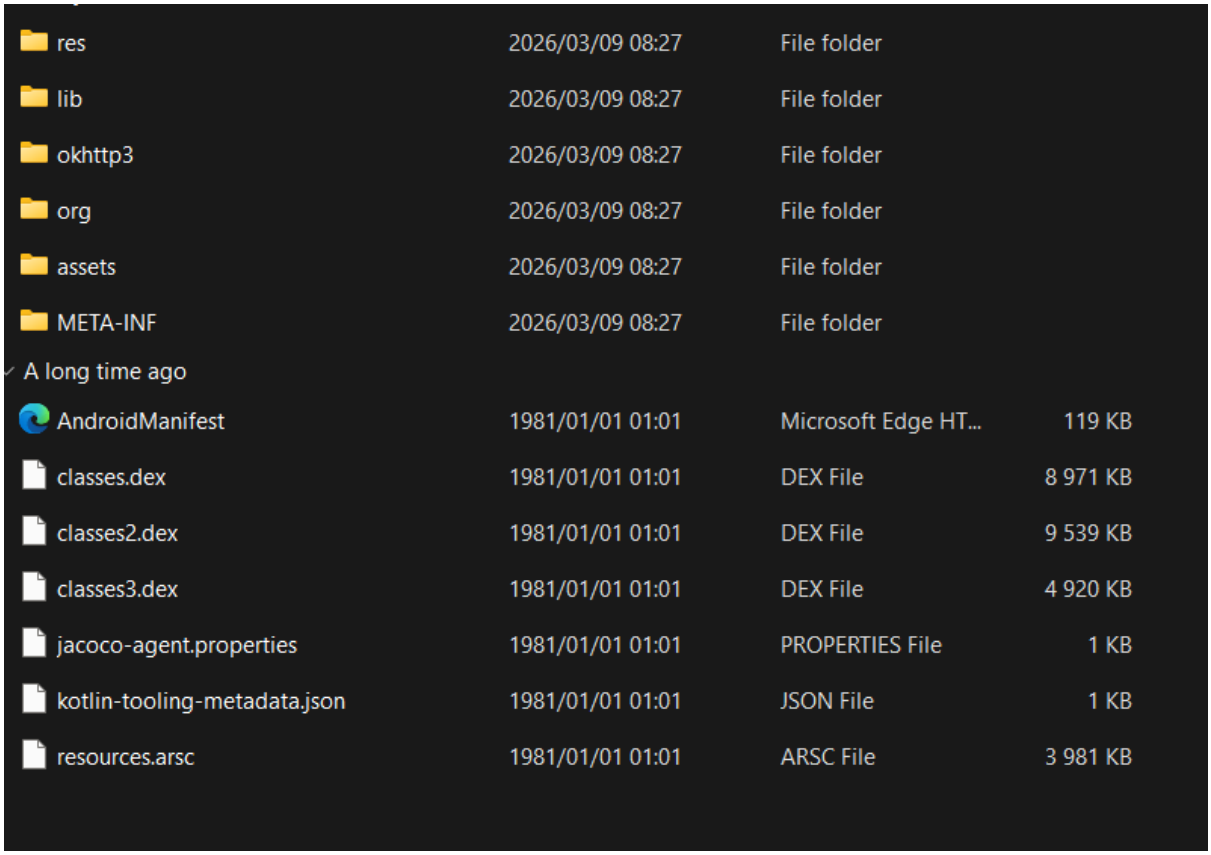


Figure 2: Extracted APK directories

The image illustrates the folder structure after extraction, highlighting binary files that are not human-readable without further processing.

In the directories there is a directories named manifest , so I moved in to it we have other files inside it , I need to use a tool called apktool to decompile the application packages into a readable formats , this is because zipping and unzipping it will provide the directories in binary formats which are not readable. We will use the command apktool d (name of the apk file):

To download an apktool I need to Github and and download the jarfile and the apktool.bat , put them in a separate folder folder and name it apktool. Place the folder on C:\apktool for easy access , Add apk tool folder into environment viriable path so you can access it with command prompt. Go to command prompt and type apktool to confirm it was added to path.

```
C:\Users\Thando Sibakhele>apktool
Apktool 2.11.1 - a tool for reengineering Android apk files
with smali 3.0.9 and baksmali 3.0.9
Copyright 2010 Ryszard Wiśniewski <brut.alll@gmail.com>
Copyright 2010 Connor Tumbleson <connor.tumbleson@gmail.com>

usage: apktool
 -advance,--advanced   Print advanced information.
 -version,--version    Print the version.
usage: apktool if|install-framework [options] <framework.apk>
 -p,--frame-path <dir>   Store framework files into <dir>.
 -t,--tag <tag>          Tag frameworks using <tag>.
usage: apktool d[ecode] [options] <file_apk>
 -f,--force              Force delete destination directory.
 -o,--output <dir>       The name of folder that gets written. (default: apk.out)
 -p,--frame-path <dir>   Use framework files located in <dir>.
 -r,--no-res             Do not decode resources.
 -s,--no-src             Do not decode sources.
 -t,--frame-tag <tag>    Use framework files tagged by <tag>.
usage: apktool b[uild] [options] <app_path>
 -f,--force-all          Skip changes detection and build all files.
 -o,--output <file>      The name of apk that gets written. (default: dist/name.apk)
 -p,--frame-path <dir>   Use framework files located in <dir>.

For additional info, see: https://apktool.org
For smali/baksmali info, see: https://github.com/google/smali
```

Figure 3 : Apktool commands

This figure shows that apk tool was successfully installed and added to path.

Now decompiled the application. For convieniences I need to make sure the app is on the apktool directory or folder , I run the command: apktool d InsecureBankv2.apk -o Insecurebank2_source

d = decompile

Insecurebankv2.apk = target app

-o Insecurebankv2_source = output folder source files

This extract resources ,manifest and smali code into this Insecurebankv2_source.

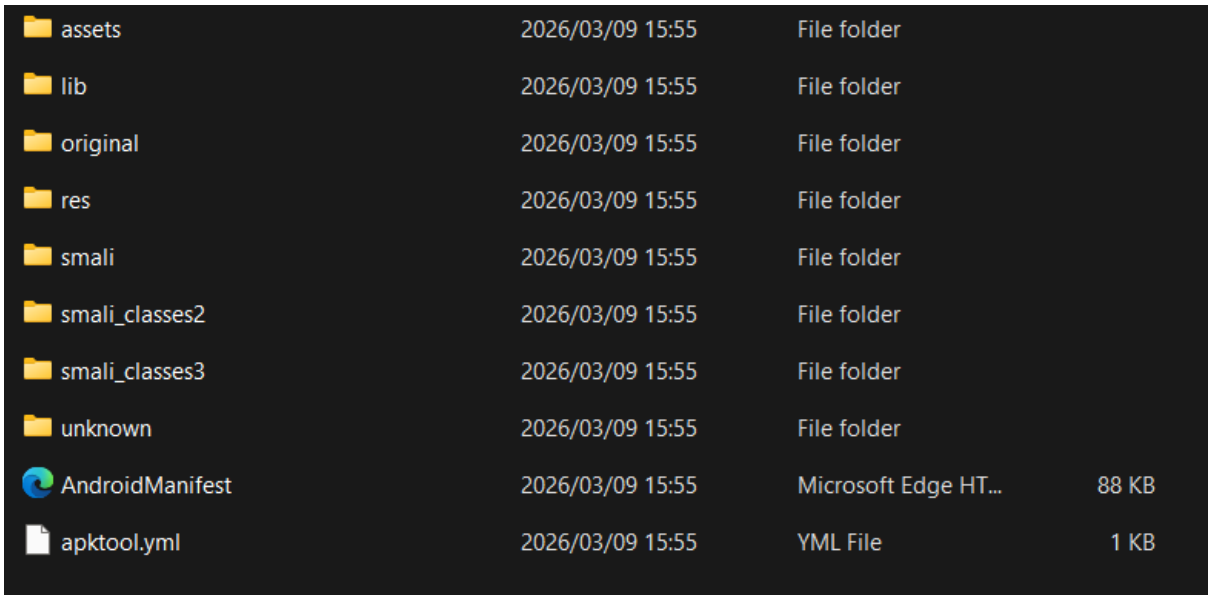


Figure 4 :shows the smali_classes which means they are now readable files and no binaries

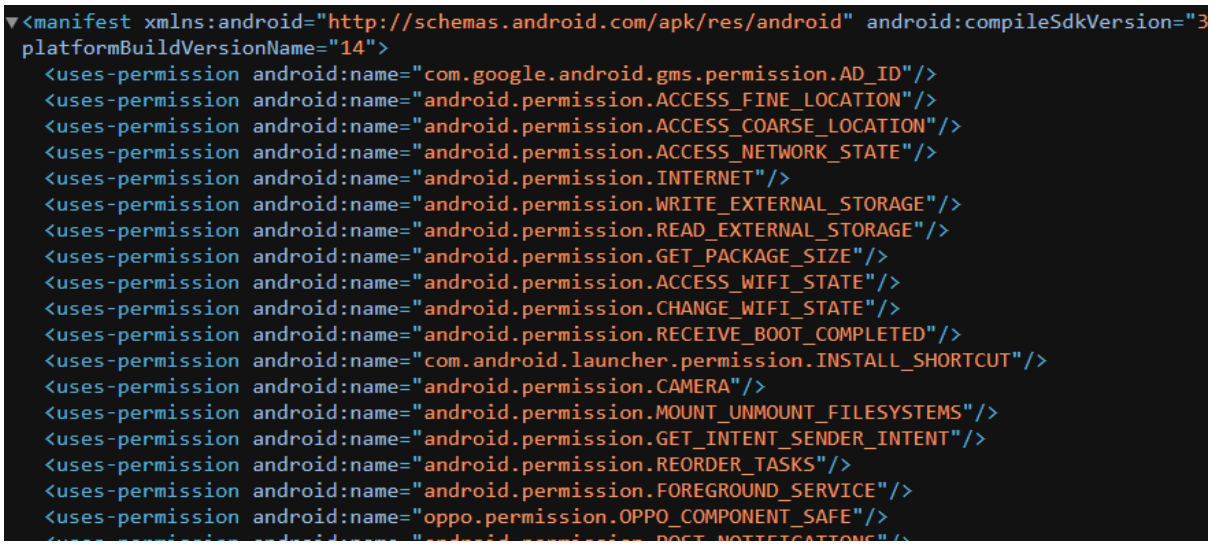


Figure 5 : Even the manifest file is in the readable format.

7. ADB COMMAND AND DEVCE INTERACTION

7.1 INSTALLING ADB

After that I need to download Android debug bridge (ADB) from the official website and then installing it to the machine , so I added ADB to system path in environment variable in order to access it through command line:

Using search and typing **environment variable** , tap **Enviroment variable** button , click **path** , then click **new** then add the path where adb was installed usually **C:\platforms-tools.**

The screenshots provide a step by step :

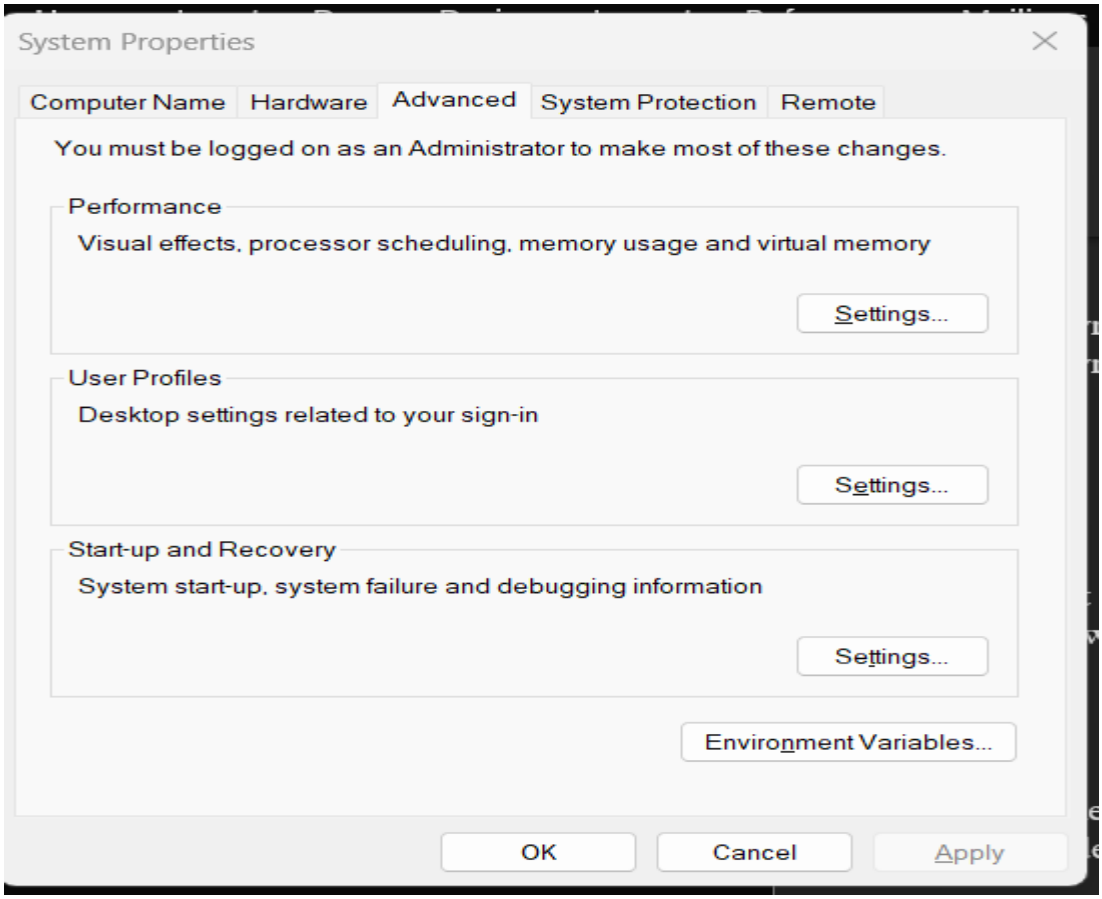


Figure 6 : Tap environment viriables

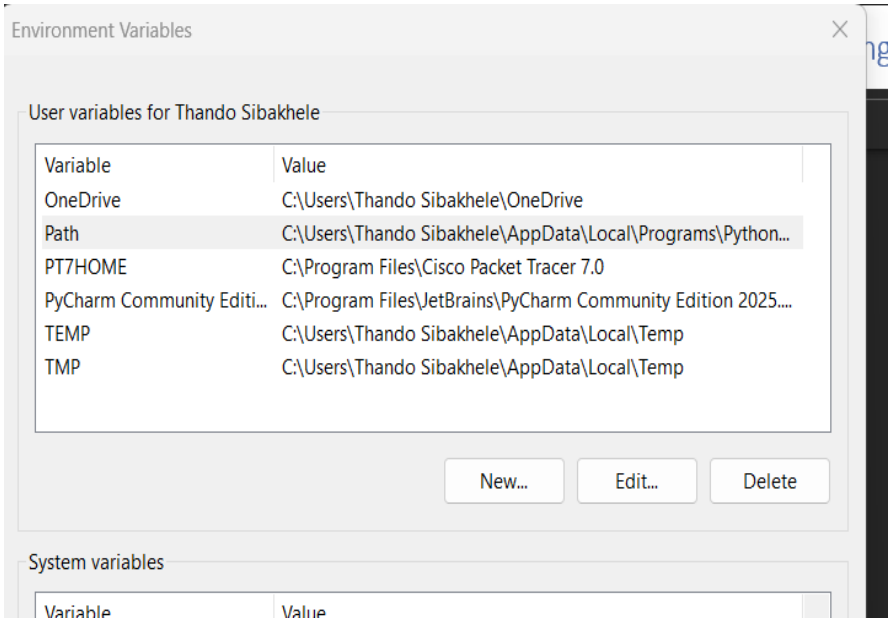


Figure 7: Tap **path** , then **edit**

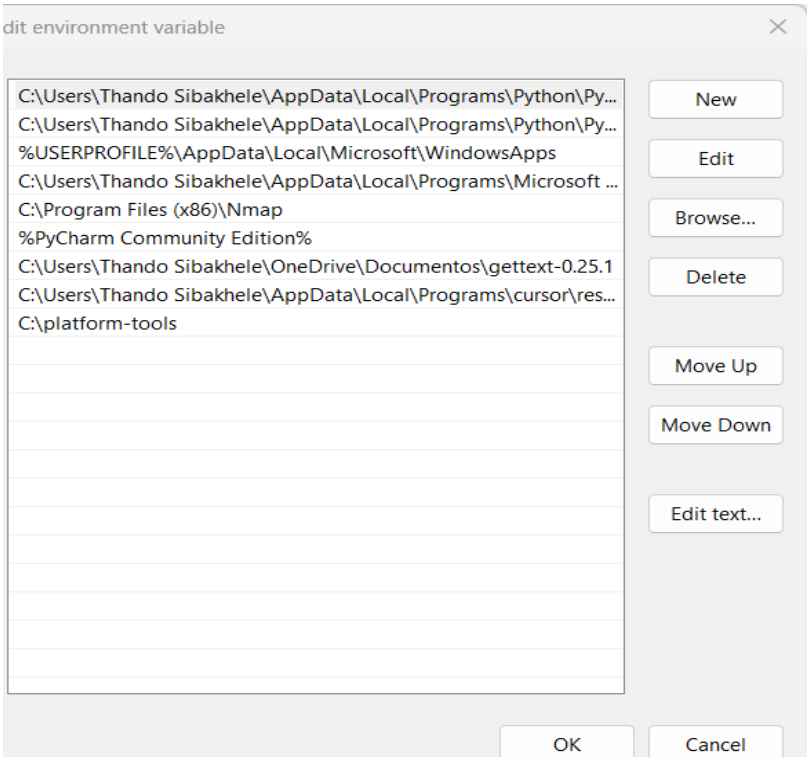


Figure 8 : Click **new** and add the the path.

I have already added the path **C:\platfrom-tools**

Now to confirm if ADB is installed and added to path use command prompt and type ADB a result similar to the screenshot below will appear:

```
C:\Users\Thando Sibakhele>adb
Android Debug Bridge version 1.0.41
Version 36.0.2-14143358
Installed as C:\platform-tools\adb.exe
Running on Windows 10.0.26100

global options:
 -a                         listen on all network interfaces, not just localhost
 -d                         use USB device (error if multiple devices connected)
 -e                         use TCP/IP device (error if multiple TCP/IP devices avai
lable)
 -s SERIAL                  use device with given serial (overrides $ANDROID_SERIAL)
 -t ID                      use device with given transport id
 -H                         name of adb server host [default=localhost]
 -P                         port of adb server [default=5037]
 -L SOCKET                  listen on given socket for adb server [default=tcp:local
host:5037]
 --one-device SERIAL|USB  only allowed with 'start-server' or 'server nodaemon', s
erver will only connect to one USB device, specified by a serial number or USB dev
ice address.
 --exit-on-write-error    exit if stdout is closed

general commands:
 devices [-l]               list connected devices (-l for long output)
 help                       show this help message
 version                    show version num
```

Figure 9 : Shows ADB command you can run which confirms ADB was installed and added to path successfully.

8. ABD COMMANDS

Now I am going to use ADB , so first we need to print out the name of the device connected and serial number , I am using genymotion to emulate the android device.

finding the device name and serial number

8.1 ADB DEVICES AND ADB GET-SERIALNO

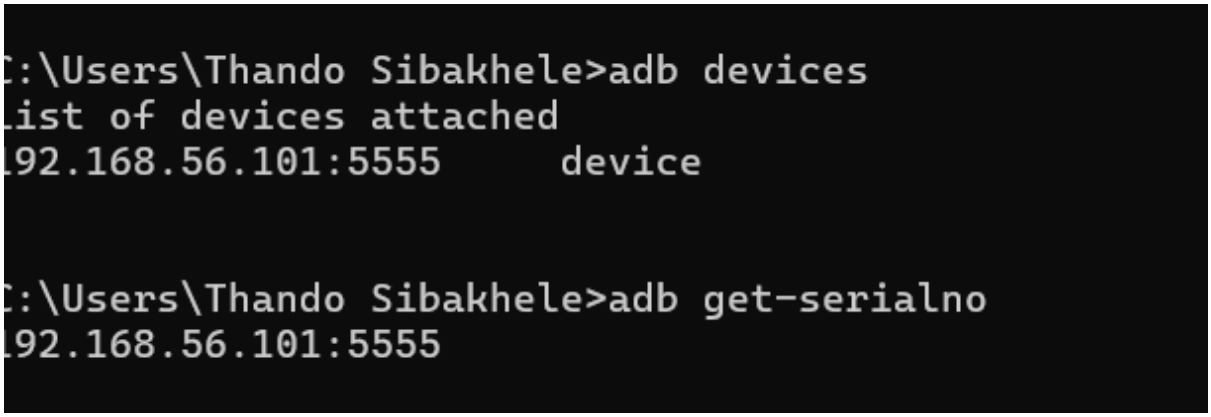


Figure 10: Shows device name and serial number using command adb devices and adb get-serno.

8.2 ADB INSTALLING APK

Installing the apk file to the phone , I will use the command adb install as shown in the screenshot:

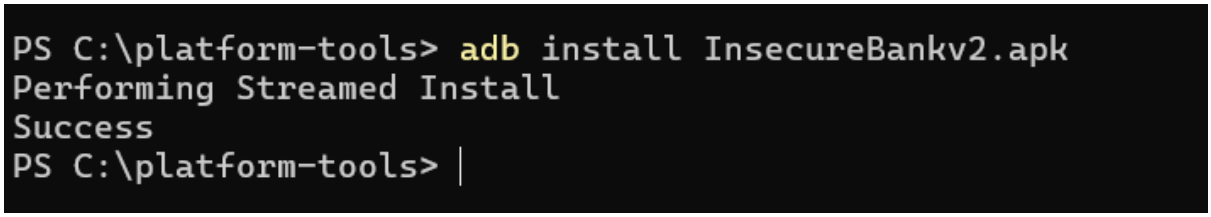


Figure 11: Shows installing success

Note: It is advisable that you paste your application on your working directory , in my instance I was on C:\platform-tools directory , if not I need to make sure I copy the app as path the paste it on the code or otherwise you will get an error.

8.3 ADB PULL AND PUSH

Now I need to do a adb push and pull on the emulated phone:

This is simply copying and extracting data to and from the phone respectivelyI

I will copy a file from the computer and retrieve it from the phone using adb:

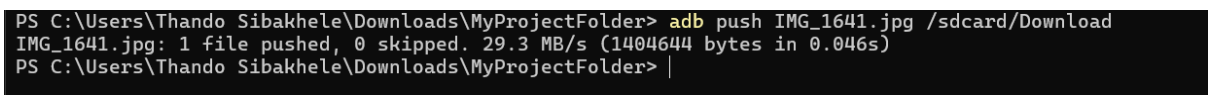


Figure 12: Shows image file copied into /sdcard/Download using adb push

This result suggest that the file was copied successfully :

Note: I always make sure the file is on the working directory for better convenience or else copy the full path of the file.

The pull command: here I am going to pull the file I just pushed above

```
PS C:\Users\Thando Sibakhele\Downloads\MyProjectFolder> adb pull /sdcard/Download C:\platform-tools
/sdcard/Download/: 2 files pulled, 0 skipped. 20.4 MB/s (1405631 bytes in 0.066s)
PS C:\Users\Thando Sibakhele\Downloads\MyProjectFolder>
```

Figure 13: Adb pull was successful

8.4 ADB GET STATE

Next step is using command adb get state , this command is used to show or return the connection state of the device ,usually output is device , offline or unauthorized. Screenshot of the adb get state command in action .

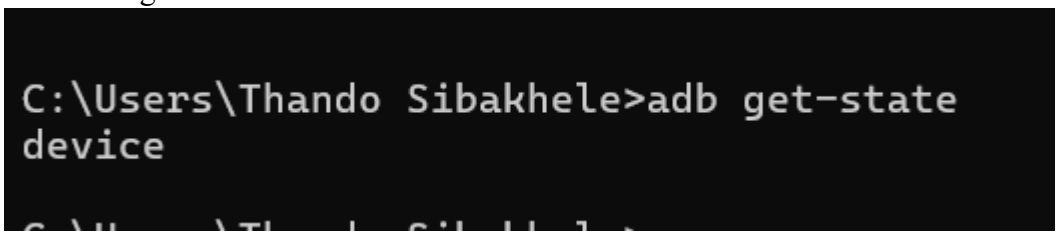


Figure 14: The output shows that the device is connected .

8.5 ADB BACKUP

Another adb command which I will look is adb backup of the apk , This is simply copying or backing up the mobile app data

```
PS C:\platform-tools> adb backup - InsecureBank.apk
WARNING: adb backup is deprecated and may be removed in a fu
ture release
Now unlock your device and confirm the backup operation...
PS C:\platform-tools>
```

Figure 15: Shows that backup was successful after confirming on the device

8.6 ADB SHELL DUMP SYS

Another command is adb shell dump sys – this command used to dump or retrieve system service info I will also provide screenshot of the command adb shell dumpsys activity which is used to show running activities , task, and app states

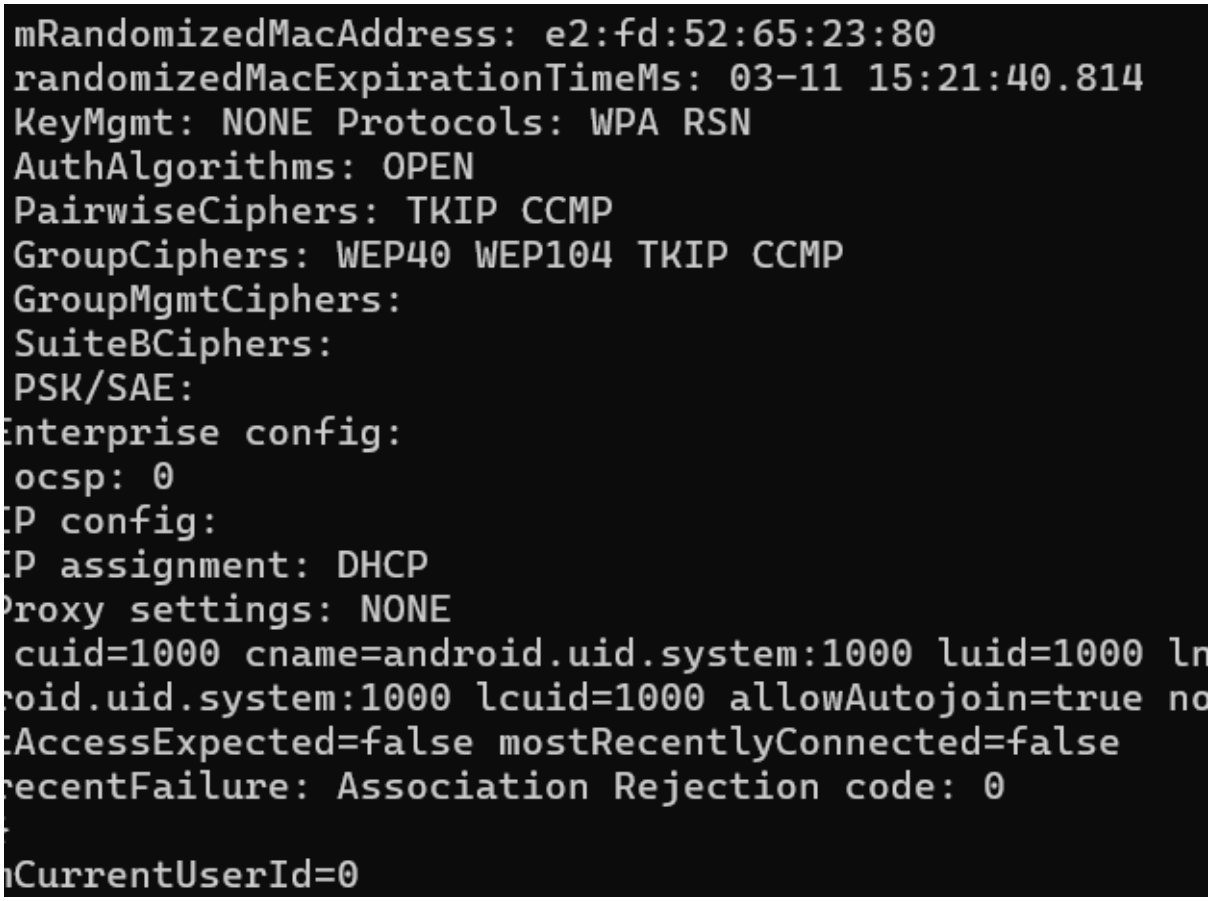


Figure 16 : The list is endless I just provided the part of the output

8.7 ADB SHELL DUMP SYS ACTIVITY

```
PS C:\platform-tools> adb shell dumpsys activity
ACTIVITY MANAGER SETTINGS (dumpsys activity settings) activi
ty_manager_constants:
  max_cached_processes=32
  background_settle_time=60000
  fgservice_min_shown_time=2000
  fgservice_min_report_time=3000
  fgservice_screen_on_before_time=1000
  fgservice_screen_on_after_time=5000
  content_provider_retain_time=20000
  gc_timeout=5000
  gc_min_interval=60000
  force_bg_check_on_restricted=true
  full_pss_min_interval=1200000
  full_pss_lowered_interval=300000
  power_check_interval=300000
  power_check_max_cpu_1=25
  power_check_max_cpu_2=25
  power_check_max_cpu_3=10
  power_check_max_cpu_4=2
  service_usage_interaction_time=1800000
  usage_stats_interaction_interval=7200000
  service_restart_duration=1000
  service_reset_run_duration=60000
  service_restart_duration_factor=4
  service_min_restart_time_between=10000
  service_max_inactivity=1800000
  service_bg_start_timeout=15000
  service_bg_activity_start_timeout=10000
  service_crash_restart_duration=1800000
```

Figure 17: shows the output of dumpsys activity command , it shows the running activity app state and tasks

8.8 ADB SHELL WM

I will also try the command adb shell wm . this command control the display size of the phone so we will use size and density, which means we will use command adb shell wm size and adb shell wm density. Wm stand for Window manager

```
PS C:\platform-tools> adb shell wm size 1080x1920
PS C:\platform-tools> adb shell wm density 480
```

Figure 18 : show commands have been successful . When I go back to the emulator phone I can see that density and Size of the phone has changed.

8.9 ADB SHELL PM LIST PACKAGES

Another code is adb shell list packages . This command list all application on the android phone. You can also modify your command by adding -3 to view third party file only , -s for system apps only , -f shows apk file path

```
PS C:\platform-tools> adb shell pm list packages
package:com.genymotion.settings
package:com.android.cts.priv.ctsshim
package:com.android.internal.display.cutout.emulation.corner
package:com.example.android.livecubes
package:com.android.internal.display.cutout.emulation.double
package:com.android.providers.telephony
package:com.android.dynsystem
package:com.android.theme.color.amethyst
package:com.android.theme.icon.pebble
package:com.android.providers.calendar
package:com.android.providers.media
package:com.android.internal.systemui.navbar.gestural_wide_b
ack
package:com.android.theme.color.sand
package:com.android.wallpapercropper
package:com.android.theme.icon.vessel
package:com.android.theme.color.cinnamon
package:com.android.theme.icon_pack.victor.settings
package:com.android.theme.icon_pack.rounded.systemui
package:com.android.theme.icon.taperedrect
package:com.android.documentsui
package:com.android.externalstorage
```

Figure 19: Shows output of the all application in the device

```
PS C:\platform-tools> adb shell pm list packages -3
package:com.android.insecurebankv2
package:com.withsecure.dz
PS C:\platform-tools> adb shell pm list packages -f
package:/system/app/SettingsService/SettingsService.apk=com.
genymotion.settings
package:/system/priv-app/CtsShimPrivPrebuilt/CtsShimPrivPreb
uilt.apk=com.android.cts.priv.ctsshim
package:/system/product/overlay/DisplayCutoutEmulationCorner
/DisplayCutoutEmulationCornerOverlay.apk=com.android.interna
l.display.cutout.emulation.corner
package:/system/app/CubeLiveWallpapers/CubeLiveWallpapers.ap
k=com.example.android.livecubes
package:/system/product/overlay/DisplayCutoutEmulationDouble
/DisplayCutoutEmulationDoubleOverlay.apk=com.android.interna
l.display.cutout.emulation.double
package:/system/priv-app/TelephonyProvider/TelephonyProvider
.apk=com.android.providers.telephony
package:/system/priv-app/DynamicSystemInstallationService/Dy
namicSystemInstallationService.apk=com.android.dynsystem
package:/system/product/overlay/AccentColorAmethyst/AccentCo
lorAmethystOverlay.apk=com.android.theme.color.amethyst
package:/system/product/overlay/IconShapePebble/IconShapePeb
```

Figure 20: Shows only third party applications

```
PS C:\platform-tools> adb shell pm list packages -s
package:com.genymotion.settings
package:com.android.cts.priv.ctsshim
package:com.android.internal.display.cutout.emulation.cor
package:com.example.android.livecubes
package:com.android.internal.display.cutout.emulation.dou
package:com.android.providers.telephony
package:com.android.dynsystem
package:com.android.theme.color.amethyst
package:com.android.theme.icon.pebble
package:com.android.providers.calendar
```

Figure 21: shows only system apk files using the -s switch

Let move on to getting the manifest of an apk file using adb commands

8.10 ADB SHELL PM PATH <APK>

First I have to adb shell pm path com.example.myapp , this will give me the path of the app. Then I have to pull the base.apk file of the app using the path I got from the above, I then need to use any apktool to decompile the base.apk file then we will be able to see the manifest file content of the apk.

```
PS C:\platform-tools> adb shell pm path com.android.insecure
bankv2
package:/data/app/~~HymVb7GM9KciePv36Iks-g==/com.android.ins
ecurebankv2-8mJMfeWNL9p2P9s0nNs2EA==/base.apk
PS C:\platform-tools>
```

Figure 22 : Shows the path to the base.apk

We got the base.apk path now we use it to pull the base.apk file

```
PS C:\platform-tools> adb pull /data/app/~~HymVb7GM9KciePv36
Iks-g==/com.android.insecurebankv2-8mJMfeWNL9p2P9s0nNs2EA==/
base.apk C:\platform-tools
/data/app/~~HymVb7GM9KciePv3.../s (3462429 bytes in 0.075s)
```

Figure 23: Shows adb pull successful

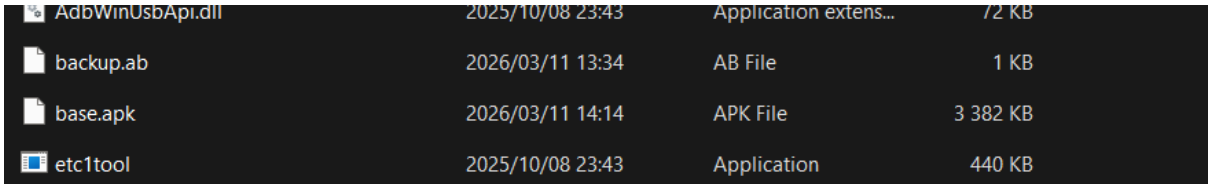


Figure 24: shows the base.apk is now on my system

```
PS C:\platform-tools> adb pull /data/app/~~HymVb7GM9KciePv36
Iks-g==/com.android.insecurebankv2-8mJMfeWNL9p2P9s0nNs2EA==/
base.apk C:\platform-tools
/data/app/~~HymVb7GM9KciePv3.../s (3462429 bytes in 0.075s)
PS C:\platform-tools> apktool d base.apk
I: Using Apktool 2.11.1 on base.apk with 5 threads
I: Baksmaling classes.dex...
I: Loading resource table...
I: Decoding file-resources...
I: Loading resource table from file: C:\Users\Thando Sibakhe
le\AppData\Local\apktool\framework\1.apk
I: Decoding values */* XMLs...
I: Decoding AndroidManifest.xml with resources...
I: Regular manifest package...
I: Copying original files...
I: Copying unknown files...
Press any key to continue . . .
```

Figure 25 : decompiling the base.apk file using apk tool as we have used it on the previous topics

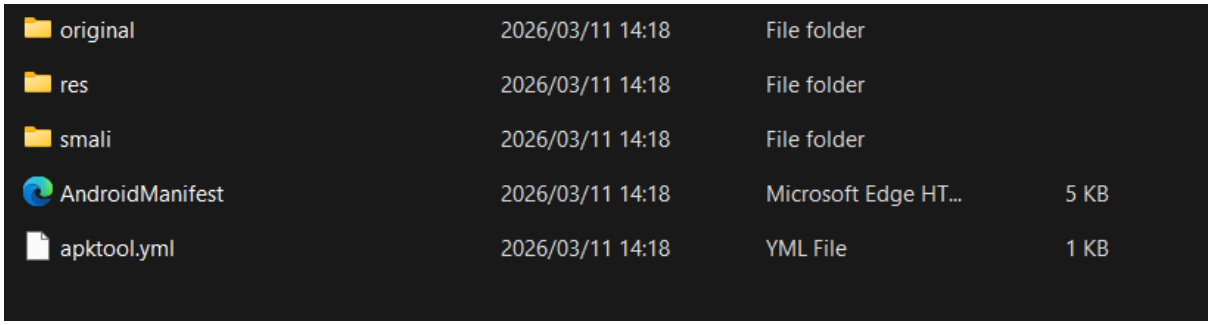


Figure 26: Now I should have a folder showing the files also the manifest file which can be Viewed through the browser.

Note: I downloaded and installed apktool and added it to path .

9. ADVANCE DECOMPILATION TECHNIQUES

Normally APK file is a packed application, So decompiling help reveal JAVA/Kotlin source code , resources and manifest.

This files help in the following.

Manifest file defines permissions, components (activities, services, receivers), and intent filters.

• By decompiling, you can check if the app requests dangerous permissions (like SMS, contacts, location) or exposes exported components that attackers could exploit.

Inspect hardcoded API keys, credentials, or endpoints.

- Identify weak crypto implementations (e.g., MD5, hardcoded salts).
- Spot insecure IPC (inter-process communication) or misconfigured WebViews.

Extract hidden strings, logs, or evidence stored in resources.

- Trace how user data is processed or transmitted.
- Useful in digital investigations to reconstruct app activity.

In the above example I used apktool to decompile an application , here I will also demonstrate how to use Dex2jar and JD-GUI

Dex2jar is used to compile the dex format of the apk to jar file in order to be readable by human.

JD_GUI – is used to view the jar file we decompiled

NOTE: all this tools need java to be installed in your system.

Downloaded java on the official installer and installed in my system.

9.1 DOWNLOADING DEX2JAR AND JD-GUI

Then download the Dex2jar tool from the official website

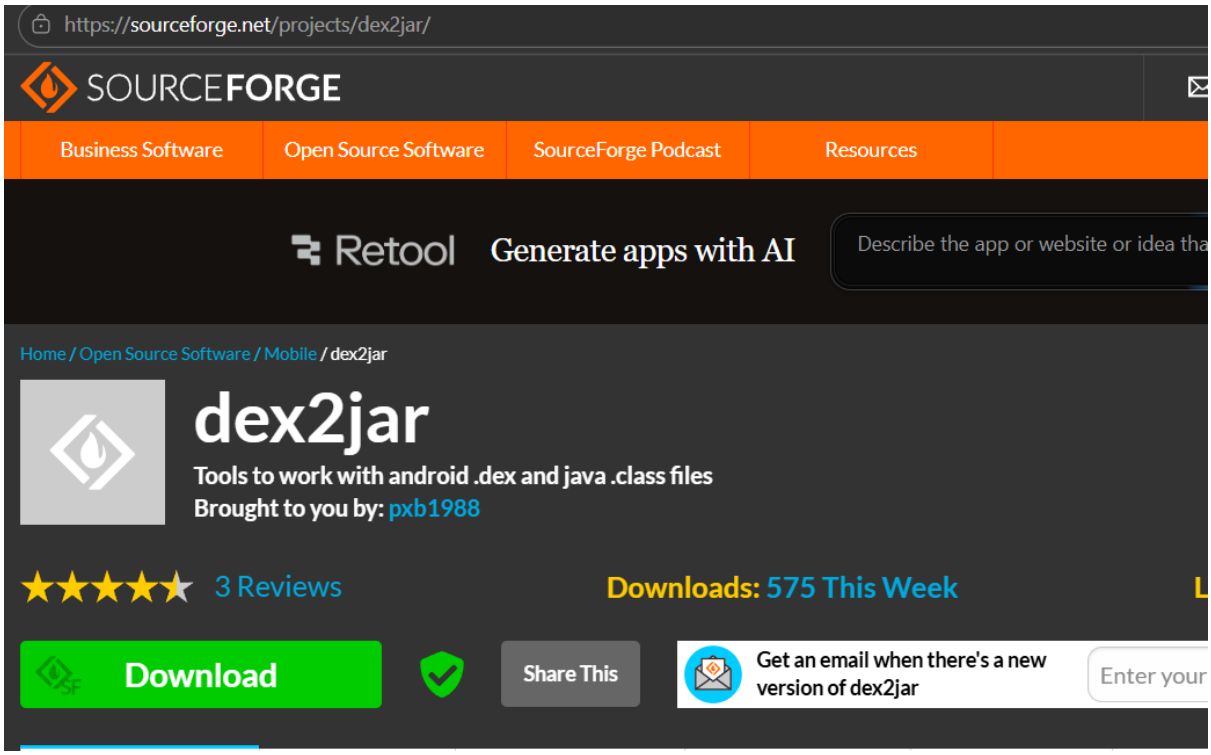


Figure 28: Shows a the official installer website for dex2jar

Also download the JD-GUI from the official website- this tool is used to view the jar file after decompiled by the dex2jar tool



Figure 29 : shows the official installer of jd-gui tool

9.2 DECOMPILING THE APP USING DEX2JAR

Now I have to use the dex2jar tool to decompile an application , for my demonstration I will use Shareit.apk file which Is widely used for sharing files among android phones

So after downloading the dex2jar tool it will come as a zipped file extract it and inside the folder where it was extracted paste your application that you want to decompile then open command prompt in that working directory and run the command on the screenshot below.

We I to run the d2j-dex2jar.bat which is the batch file followed by the name of the application

```
C:\Users\Thando Sibakhele\Downloads\MyProjectFolder\dex-tools-v2.4\dex-tools-v2.4>d2j-dex
2jar.bat SHAREit.apk
dex2jar SHAREit.apk -> .\SHAREit-dex2jar.jar
WARN: ignored invalid inner class name , treat as anonymous inner class.
WARN: ignored invalid inner class name , treat as anonymous inner class.
```

Figure 30 : shows the command in action

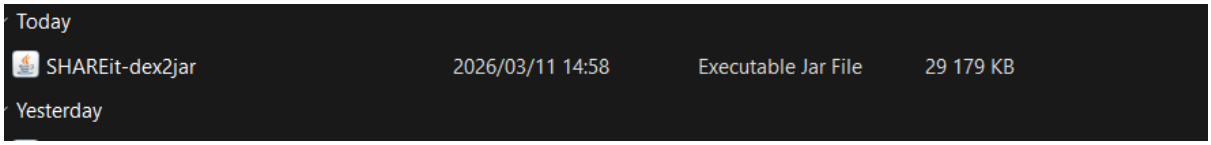


Figure 31: A file named SHAREit-dex2jar file will appear on the folder which means it has been decompiled to .jar file which is viewable by humans

9.3 USING JD_GUI

Launch the jd-gui.jar tool

In the GUI tap File – open file- navigate to the SHAREit-de2jar.jar and open it.

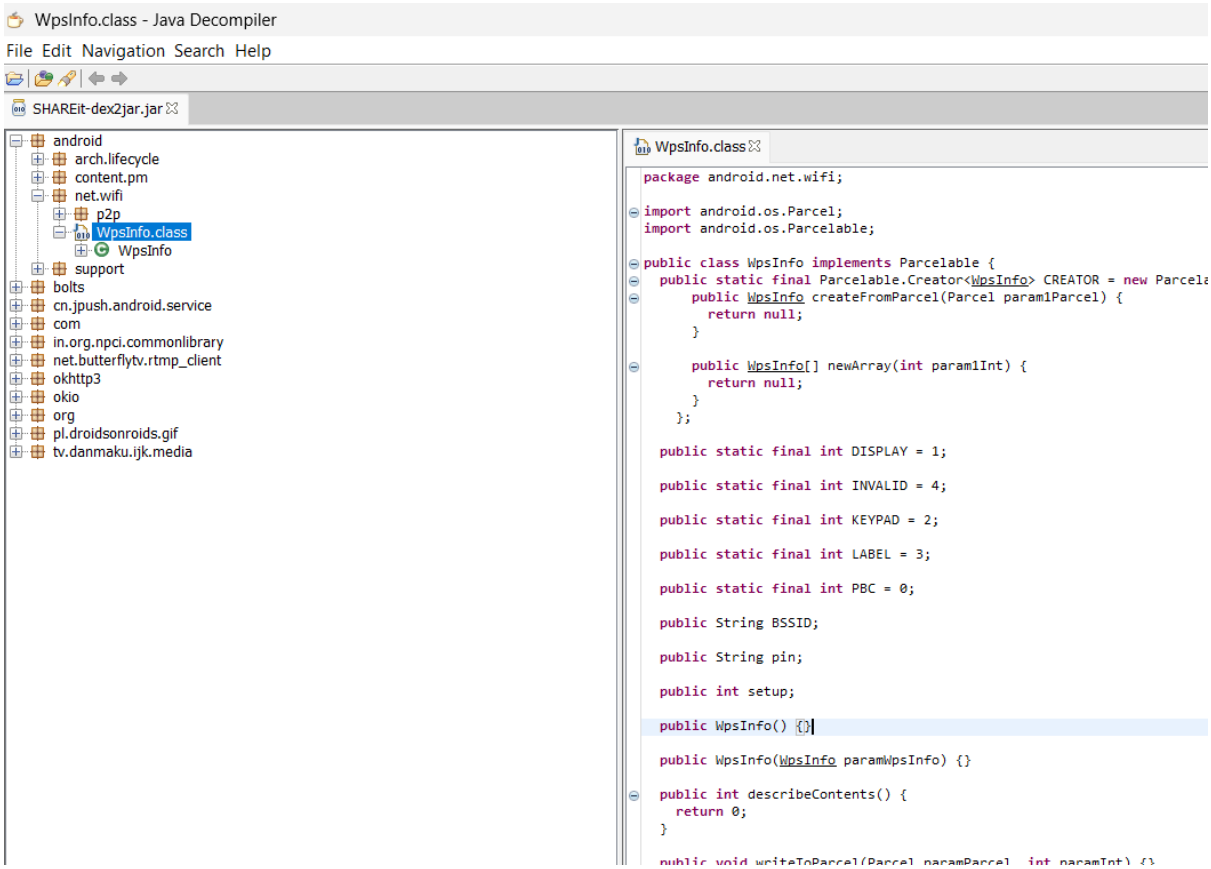


Figure 32: showing all the java code on plain text such that it is readable by humans.

9.4 INSTALLING AND USING JADX

Another tool I am going to use is JADX which does both decompiling and viewing the source java code

Download JADX from the official installer preferably downloading the one saying "with-jre-wip" as shown in the screenshot.

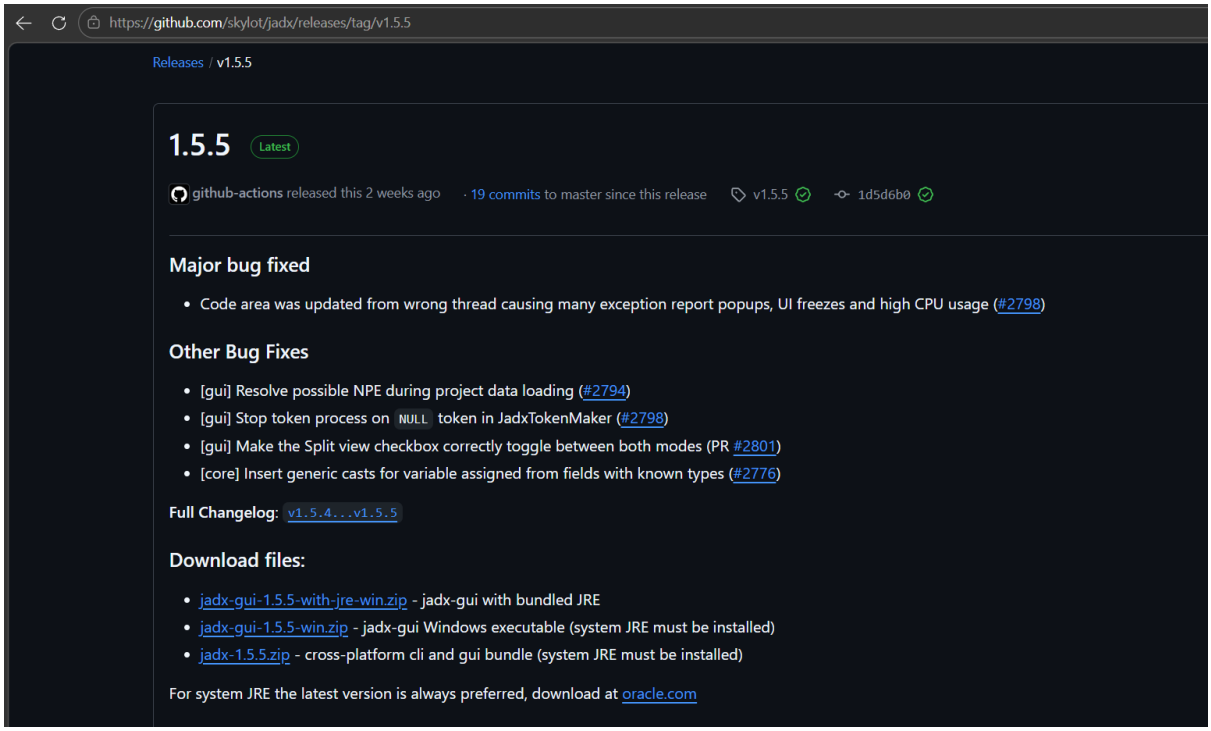


Figure 33: Shows the jadx file on the official installer which is Github

After downloading the zip file, unzip it , navigate to the folder and run the JADX application

Tap file- open file and open the file or the apk to decompile , for this demo I will use another apk called VidMate.apk which is used to download music and videos on android devices.

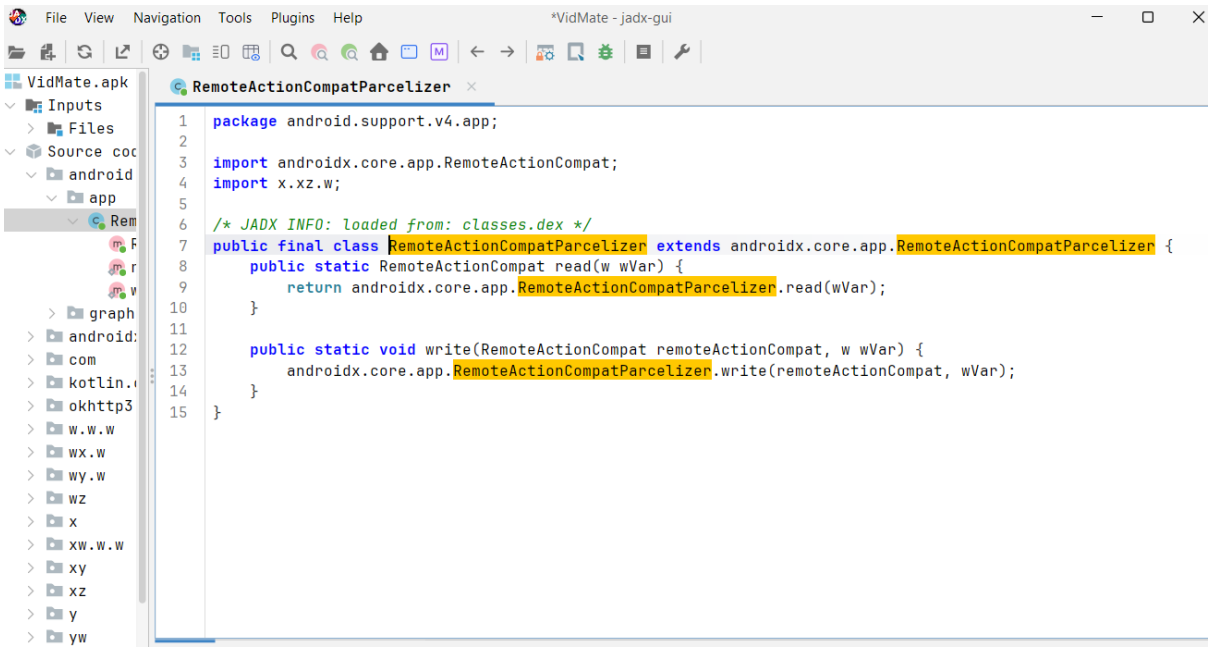


Figure 34: shows that we can view java codes in plain text

10. DYNAMIC ANALYSIS USING DROZER

10.1 INSTALLING DROZER

Another tool we will be using is drozer.

Drozer is an open tool that allows a pen tester to search for security vulnerabilities in apps and devices by assuming the role of an app and interacting with the Delvik VM , other app IPC endpoints and the underlying OS during runtime

So I will download drozer 3.1, I made sure python 3 + is installed inside my machine

Open cmd and type python --version

```
C:\Users\Thando Sibakhele>python --version
Python 3.8.5

C:\Users\Thando Sibakhele>
```

Figure 35 : shows python is installed

Normally when installing python pip comes with python, no need to install pip

Now I need to go to github webpage and download the latest version (of which it was 3.1 at the time of writing this project) and download the zip file. Extract the zip file and move to the drozer folder or directory after extraction .

Inside the drozer folder open command prompt or the terminal to install the dependencies drozer needs.

Type the following command

1. Pip install protobuf
2. Pip install pyopenssl
3. Pip install twisted

```
C:\Users\Kuma>pip install protobuf
DEPRECATION: Python 2.7 will reach the end of its life on January 1st, 2020. Please upgrade your Python as Python 2.7 wo
n't be maintained after that date. A future version of pip will drop support for Python 2.7. More details about Python 2
 support in pip, can be found at https://pip.pypa.io/en/latest/development/release-process/#python-2-support
Collecting protobuf
  Using cached https://files.pythonhosted.org/packages/27/9c/ef816295b4b40298fd0a17bf8f0ba6cf3e0c44cb2ce72257168e09996b8
b/protobuf-3.11.3-py2.py3-none-any.whl
Requirement already satisfied: setuptools in c:\python27\lib\site-packages (from protobuf) (41.2.0)
Collecting six>=1.9 (from protobuf)
  Using cached https://files.pythonhosted.org/packages/65/eb/1f97cb97bfc2390a276969c6fae16075da282f5058082d4cb10c6c5c1db
a/six-1.14.0-py2.py3-none-any.whl
Installing collected packages: six, protobuf
Successfully installed protobuf-3.11.3 six-1.14.0
WARNING: You are using pip version 19.2.3, however version 20.0.2 is available.
You should consider upgrading via the 'python -m pip install --upgrade pip' command.
```

Figure 36: Installing protobuf

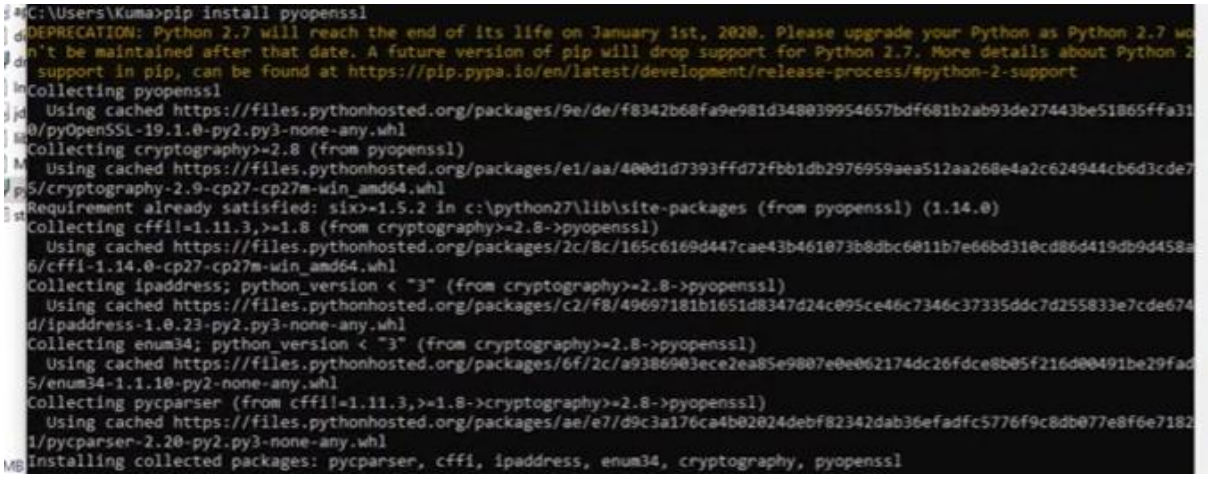


Figure 37: installing pyopnssl

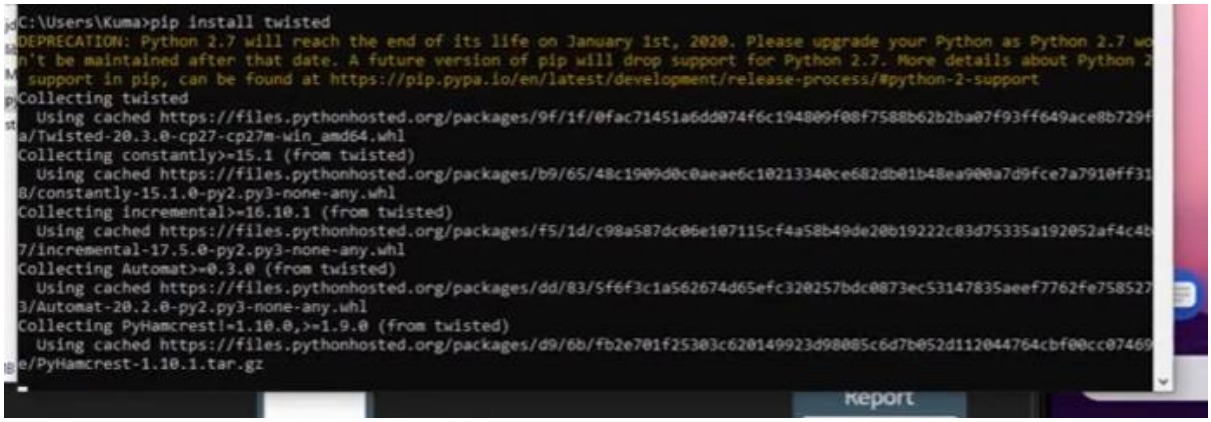


Figure 38: installing twisted

NOTE: sometimes when you run drozer for the first time you need to install another dependency : pip install service_identity . I already installed it

The last command is pip install drozer

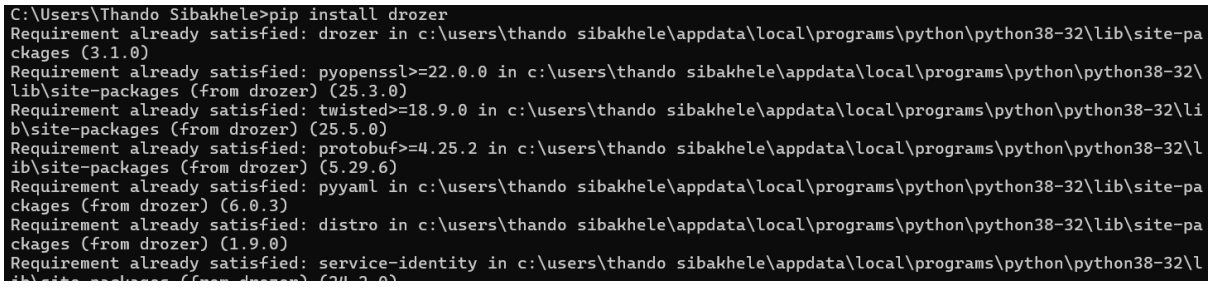


Figure 39: Drozer installation

I have already downloaded drozer as shown I the screenshot.

After completing run drozer to see if it was successfully installed

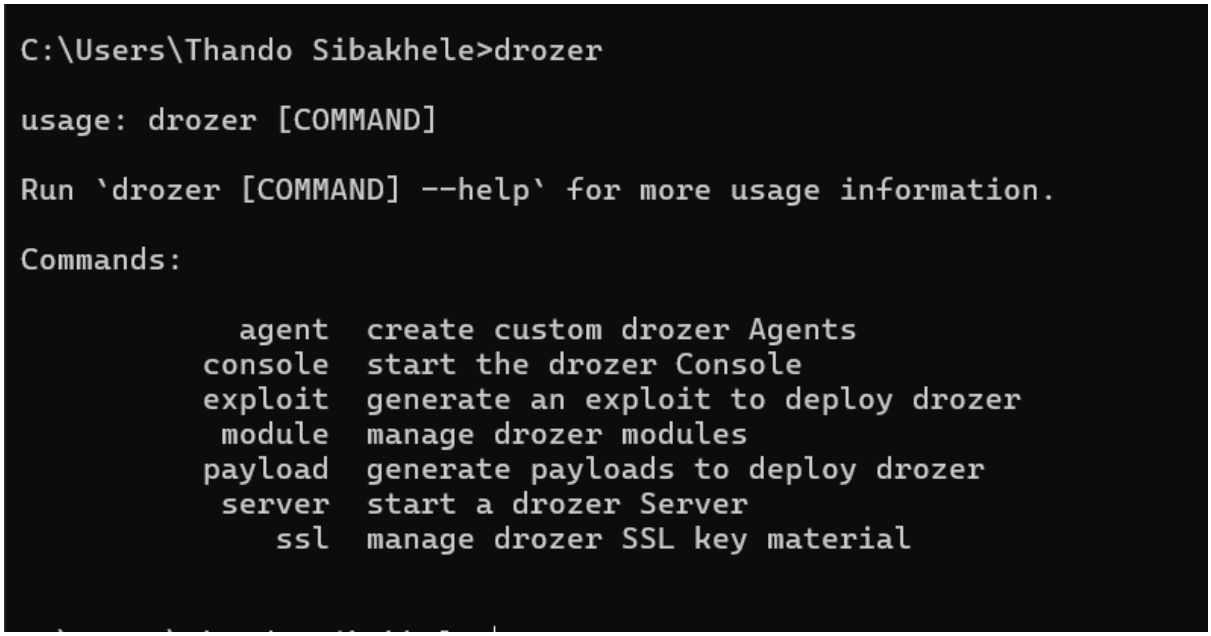


Figure 40: Shows drozer command we can run which means drozer was installed successful.

Going back to my browser and download drozer application/agent , this app will be installed in the mobile phone where I will perform our testing, this allows us to communicate with the mobile phone like a client and a server.

There are many ways :

1. Download the app on your pc and use adb install
2. Download the app direct from your phone browser.

I used the first method

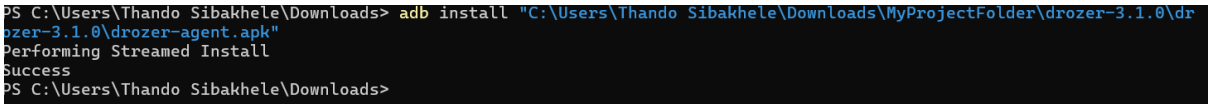


Figure 41 : Shows installing was successful on the device

After downloading open the app and I need make sure Embeded server is turned on



Figure 42: Shows embedded server on and waiting to connect , we are good to go.

Back on the terminal I need to run the following commands to connect to the agent app on the phone

adb forward tcp:31415 tcp:31415

drozer console connect

```
C:\Users\Thando Sibakhele> adb forward tcp:31415 tcp:31415
31415
```

Figure 43: First command successful as you can see it is returning the port 31415

```
C:\Users\Thando Sibakhele>drozer console connect
c:\users\thando sibakhele\appdata\local\programs\pyth
ou are using cryptography on a 32-bit Python on a 64-
ster if you switch to using a 64-bit Python.
  from cryptography.hazmat.bindings.openssl.binding i
Selecting 55b8452a67cac927 (Google AOSP on IA Emulato

           ..                    ..:.
          ..o..                  .r..
           ..a..  . ..... .  ..nd
             ro..idsnemesisand..pr
              .otectorandroidsneme.
           .,sisandprotectorandroids+.
         ..nemesisandprotectorandroidsn:.
        .emesisandprotectorandroidsnemes..
      ..isandp,..,rotecyayandro,..,idsnem.
      .isisandp..rotectorandroid..snemisis.
      ,andprotectorandroidsnemisisandprotec.
     .torandroidsnemesisandprotectorandroid.
     .snemisisandprotectorandroidsnemesisan:
     .dprotectorandroidsnemesisandprotector.

drozer Console (v3.1.0)
dz>
```

Figure 44: The output should be like this telling us that we in the drozer console where we will do out testing.

Now I am getting to the funny part , as I have mentioned above that I have installed insecurebankv2.apk for our testing which is our insecure banking app from github. So here I need to see if it is really installed and see other apps that are installed.

```
com.google.android.syncadapters.contacts (Google Contacts Sync
com.android.insecurebankv2 (InsecureBankv2)
com.android.keychain (Key Chain)
com.android.chrome (Chrome)
com.google.android.packageinstaller (Package installer)
com.google.android.gms (Google Play services)
com.google.android.gsf (Google Services Framework)
com.google.android.ims (Carrier Services)
```

Figure 45: Shows the output of the adb shell pm list packages , shows insecurebackv2 installed and have a name “com.android.insecurebankv2” I will use this name in our testing whenever we call it

10.2 RETRIVING PACKAGE INFORMATION

I will use command : run app.package.info -a com.android.insecurebankingv

```
dz> run app.package.info -a com.android.insecurebankv2
Attempting to run shell module
Package: com.android.insecurebankv2
  Application Label: InsecureBankv2
  Process Name: com.android.insecurebankv2
  Version: 1.0
  Data Directory: /data/user/0/com.android.insecurebankv2
  APK Path: /data/app/com.android.insecurebankv2-_76qCAlqH9QOvrPh6gXAlg==/base.apk
  UID: 10088
  GID: [3003]
  Shared Libraries: [/system/framework/org.apache.http.legacy.boot.jar]
  Shared User ID: null
  Uses Permissions:
  - android.permission.INTERNET
  - android.permission.WRITE_EXTERNAL_STORAGE
  - android.permission.SEND_SMS
  - android.permission.USE_CREDENTIALS
  - android.permission.GET_ACCOUNTS
  - android.permission.READ_PROFILE
  - android.permission.READ_CONTACTS
  - android.permission.READ_PHONE_STATE
  - android.permission.READ_CALL_LOG
  - android.permission.ACCESS_NETWORK_STATE
  - android.permission.ACCESS_COARSE_LOCATION
  - android.permission.READ_EXTERNAL_STORAGE
  Defines Permissions:
  - None
```

Figure 46: Shows the app permission on the device

10.3 IDENTIFYING PACKAGES ATTACK SURFACE

I will run command run.app.package.attacksurface com.android.insecurebankv2

```
dz> run app.package.attacksurface com.android.insecurebankv2
Attempting to run shell module
Attack Surface:
  5 activities exported
  1 broadcast receivers exported
  1 content providers exported
  0 services exported
    is debuggable
```

Figure 47: shows that there are :

5 activities exported

Activities are screens or entry points in the app. If they’re exported, it means other apps (or attackers) can directly launch them without needing permission. This can expose sensitive functionality if not properly secured.

1 broadcast receiver exported

Broadcast receivers listen for system-wide messages (like incoming SMS, battery changes, etc.). An exported receiver can be triggered by any app, potentially allowing malicious input or actions if not validated.

1 content provider exported

Content providers manage structured data (like databases). If exported, other apps can query or modify this data. If not protected with permissions, it could lead to data leakage or tampering.

0 services exported

Services run background tasks. None are exported here, which is safer — it means attackers can't directly bind to or start background processes.

is debuggable

This is a big red flag. A debuggable app allows attackers to attach a debugger, inspect memory, and potentially bypass security controls. Production apps should *never* be debuggable.

For a Secure application There should few or no exported services, no exported broadcast receivers no exported content providers and services and it should not be debuggable.

10.4 EXPLOITING ACTIVITIES

I use command run app.activity.info com.android.insecurebankv2

```
dz> run app.activity.info -a com.android.insecurebankv2
Attempting to run shell module
Package: com.android.insecurebankv2
  com.android.insecurebankv2.LoginActivity
    Permission: null
  com.android.insecurebankv2.PostLogin
    Permission: null
  com.android.insecurebankv2.DoTransfer
    Permission: null
  com.android.insecurebankv2.ViewStatement
    Permission: null
  com.android.insecurebankv2.ChangePassword
    Permission: null

dz>
```

Figure 48 : shows that there are certain activities such as LiginActivity but Persmission is equals to null . This means **none of these activities require special permissions to be launched**. In other words, they are **exported activities** that can potentially be accessed directly by another app or attacker, without authentication or authorization.

10.5 EXPLOITING PROVIDER INFO

we will use commands run app.provider.info -a <package name> , run scanner.provider.sqltables -a <package name>

run scanner.provider.injection -a <package name>

```
dz> run scanner.provider.sqltables -a com.android.ins
ecurebankv2
Attempting to run shell module
Scanning com.android.insecurebankv2...
Accessible tables for uri content://com.android.insec
urebankv2.TrackUserContentProvider/trackerusers/:
  android_metadata
  names
  sqlite_sequence

Accessible tables for uri content://com.android.insec
urebankv2.TrackUserContentProvider/trackerusers:
  android_metadata
  names
  sqlite_sequence
```

Figure 49:

11. DYNAMIC ANALYSIS WITH BURP SUITE

11.1 PROXY INTERCEPTION SETUP

Now lets dive into dynamic testing where by I will configure burp suite .

But first I need to export the burp suite certificate and install it to the android phone so that they can be able to communicate to each other.

Open burp suite , navigate to proxy tab , then proxy settings .

From there configure the proxy ip address and port for burp to listen on .

Then tab export/import CA certificate. Then follow the prompt to save the certificate . The certificate will be saved as .der extension .

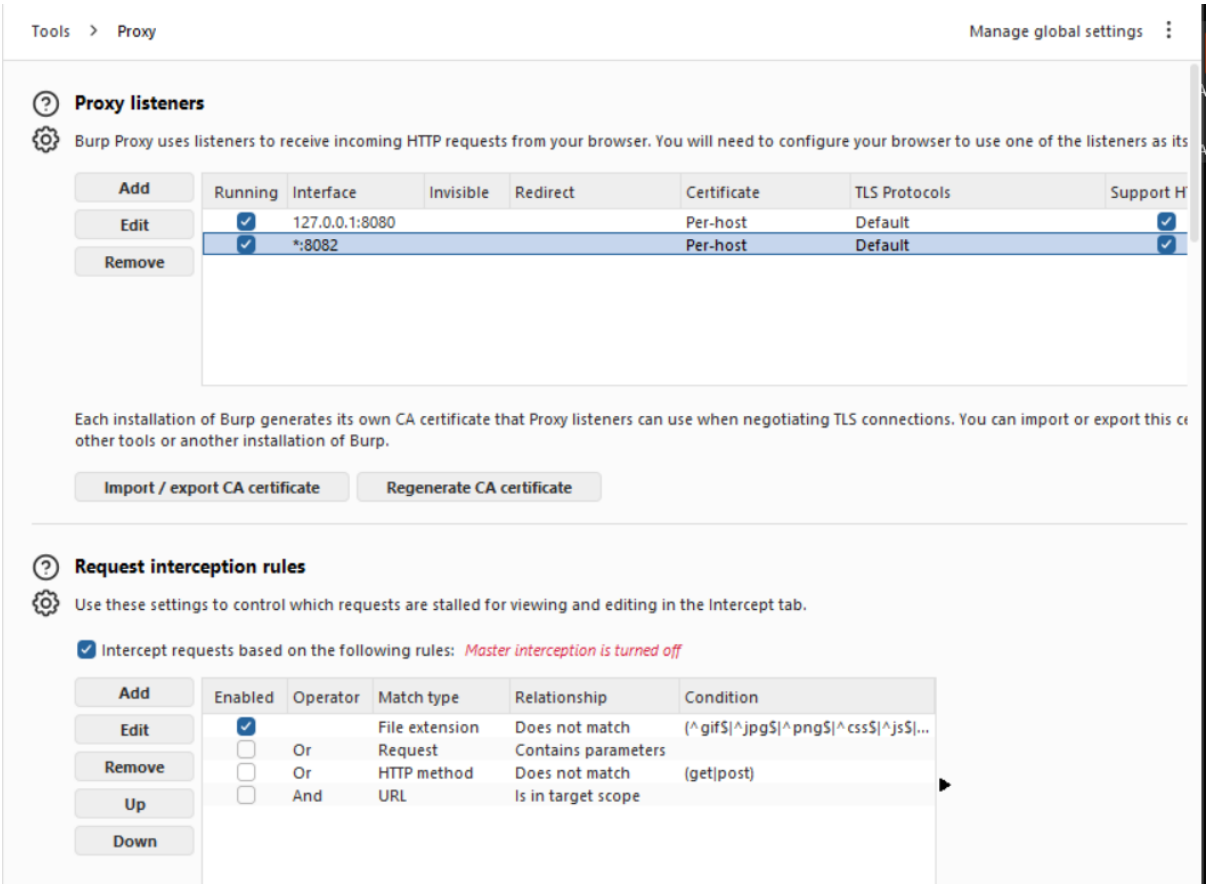


Figure 50: shows where to set the proxy and export the ca certificate.

Now since the certificate is saved as burp.der(burp is the name you specified when saving)

I need to change it to pem then generate a hexadecimal for android to be able to read it . Use the command in the screenshot:

NOTE: I need to have openssl installed on my system, install openssl from the official installer

```
C:\Users\Thando Sibakhele\Downloads>openssl x509 -inform DER
-in burp.der -out burp.pem

C:\Users\Thando Sibakhele\Downloads>
```

Figure 51: shows the certificate changed to burp.pem from burp.der

Now I need to generate the hash for the key

```
C:\Users\Thando Sibakhele\Downloads>openssl x509 -inform PEM
-subject_hash_old -in burp.crt
9a5ba575
-----BEGIN CERTIFICATE-----
MIID1zCCAr+gAwIBAgIFALJZPhUwDQYJKoZIhvcNAQELBQAwgYoxFDASBgNVB
AYT
C1BvcnRTd2lnZ2VyMRQwEgYDVQQIEwtQb3J0U3dpZ2dlcjEUMBIGA1UEBxMLU
G9y
dFN3aWdnZXIxFDASBgNVBAoTC1BvcnRTd2lnZ2VyMRcwFQYDVQQLEw5Qb3J0U
```

Figure 52: suggest the hash has been generated at the very first line (9a5ba575)

Copy that line and rename it to 9a5ba575.0 . This is because android only read a certificate in .o extension use the command below

```
C:\Users\Thando Sibakhele\Downloads>rename burp.crt 9a5ba575.
0
```

Figure 53: Shows renaming of the certificate to .0 format

Now I need to push the certificate but first I need to run **adb root** and **adb remount** , this command allows us to write on the root directory . On a rooted Phone this command will work smoothly but some phone will not be able to run adb remount according to my project there are two option to bypass that on android phone .

Option 1

Firstly, I need to navigate to the emulator directory and open command prompt ,

Type exactly this command . This command help us to tell the device to boot the emulator with a writable /system which allows you to install the certificate .

Make sure the emulator is shutdown before running the command

Pixel_3a_XL → name of the emulator or the device

The **-writable-system** →makes /system writable

The **-selinux permissive** → Releases the SELinux enforcement.

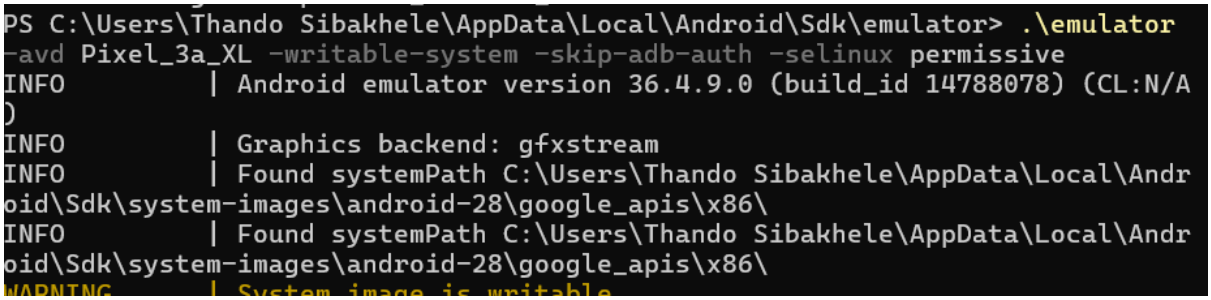


Figure 54: This command will, if it work it will power on the device with a writable system , in short it root the phone temporally

Then I should be able to copy the file or the certificate to the root directory

Note run this command on cmd as administrator , then open another tab or cmd windows to push the certificate.

I will get to how to push the certificate after we went through the second option.

Option 2

Using magisk

First thing I need to install root AVD into our machine I need to install it using git which means we need to install git for windows first

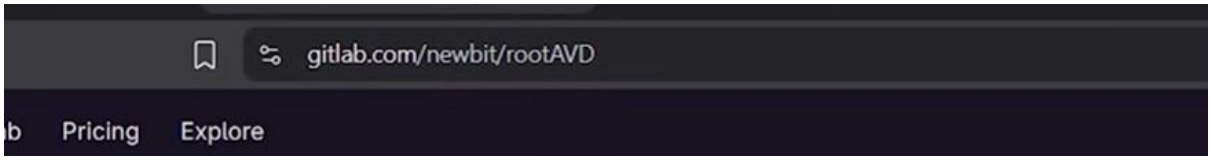
Open command prompt and type



Figure 55: Installing git into the system

After that git will be installed into our computer , moving on to installing root AVD .

Navigate to the gitlab website which is official installer of rootAVD to download or copy the link to download using git



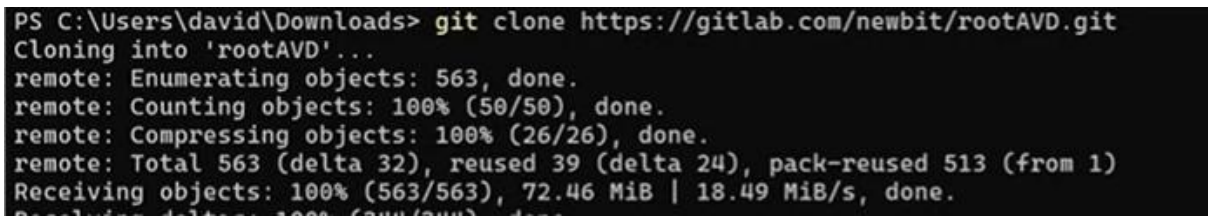


Figure 56: Downloading rootAVD into the system

Now we need to Navigate to the rootAVD directory to run the rootAVD.bat

Open command prompt in the rootAVD directory .and make sure your android device is connected



Figure 56: Running the above command to list all avd devices , look for your device you want to root.

Look at the screenshot.

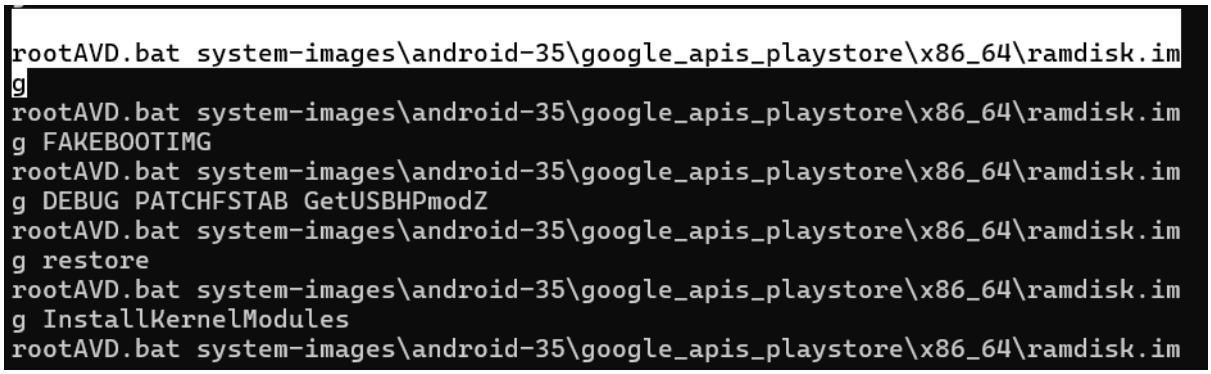


Figure 57: List of the avd devices .

Copy then line I have highlighted on the screenshot then run the command . then paste it to run the command.

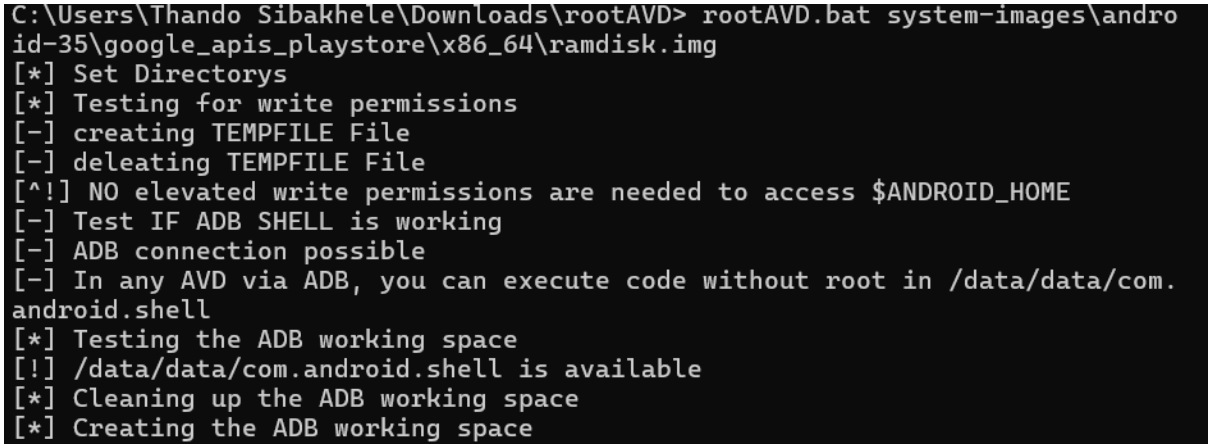


Figure 58: Paste it on the terminal and press enter.

The Rooting process will start and it will shutdown your emulator , Open or power it on then if it was successful it should show a magisk application installed .

Follow the prompt until you get the something like this one on the screenshot

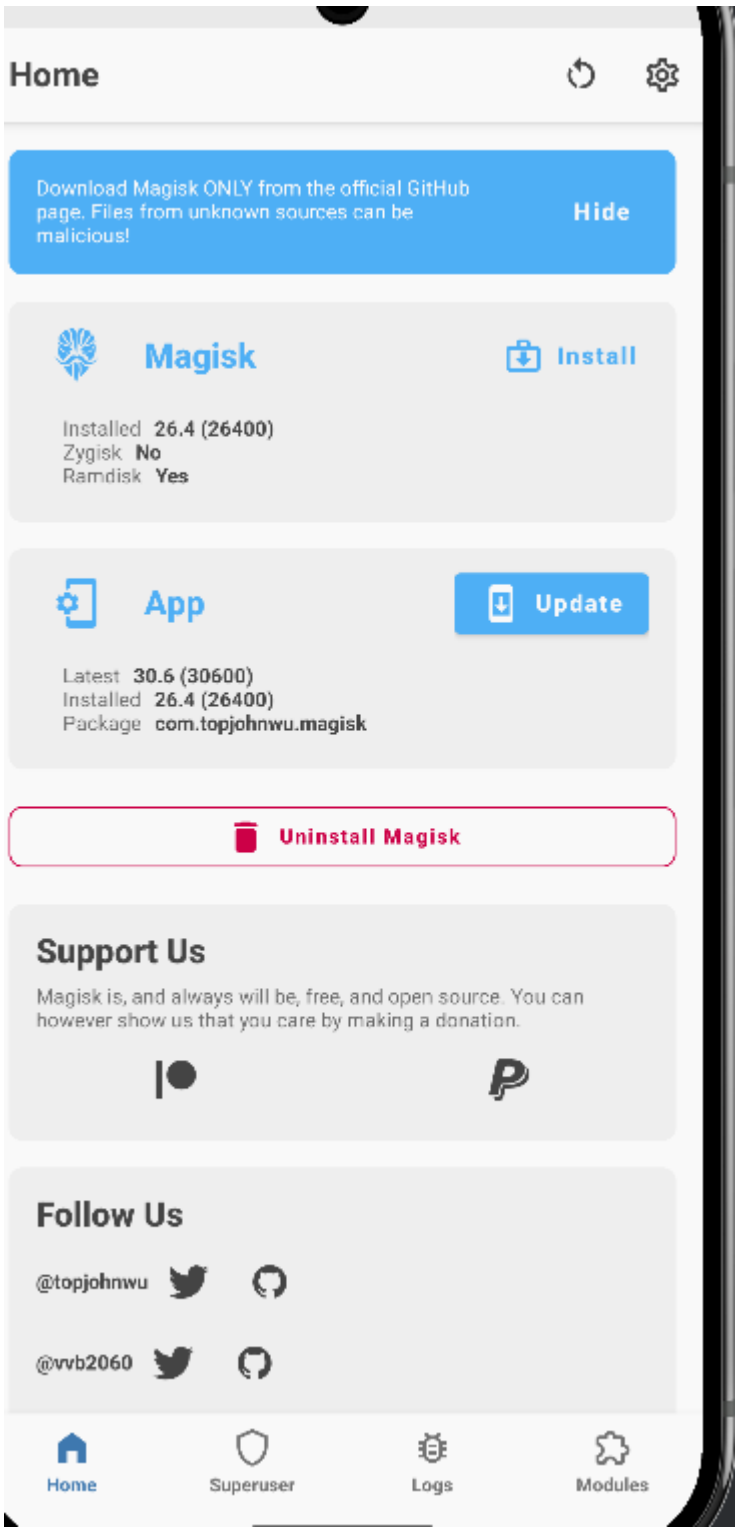


Figure 59: shows that we have successfully rooted this phone and now we need to push the certificate the /system directory.

Navigating to the directory where I save you 9a5bz575.o certificate and open command prompt. We need to run adb root the adb remount as I have mentioned earlier. Use command adb push 9a5bz575.o /system/etc/security/cacerts/ as the screen shot below

```
PS C:\Users\Thando Sibakhele\Downloads> adb root
restarting adbd as root
PS C:\Users\Thando Sibakhele\Downloads> adb remount
remount succeeded
PS C:\Users\Thando Sibakhele\Downloads> adb push 9a5ba575.0 /system/etc/security/cacerts/
9a5ba575.0: 1 file pushed, 0 skipped. 2.5 MB/s (1414 bytes in 0.001s)
PS C:\Users\Thando Sibakhele\Downloads>
```

Figure 60 : shows the command adb root and adb remount and lasty adb push to push the certificate the system directory .

On the phone navigate to settings→ security→ trusted credentials. Under system certificates we should see PortSwigger which is the certificate we installed

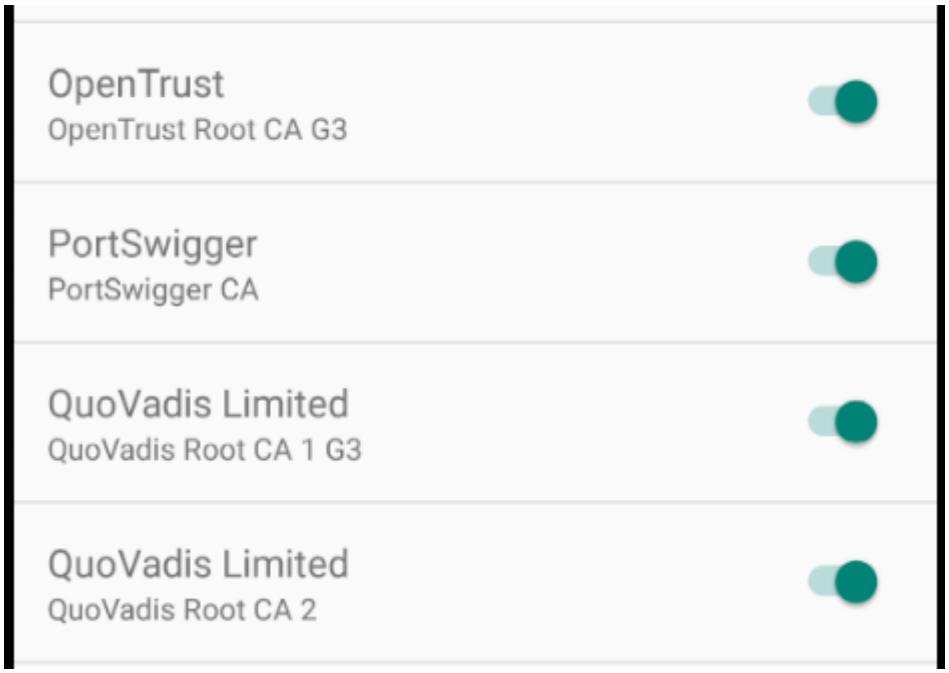


Figure61 : Show PortSwigger on the device which means certificate is installed.

Now we need to configure burp to listen in the device traffic

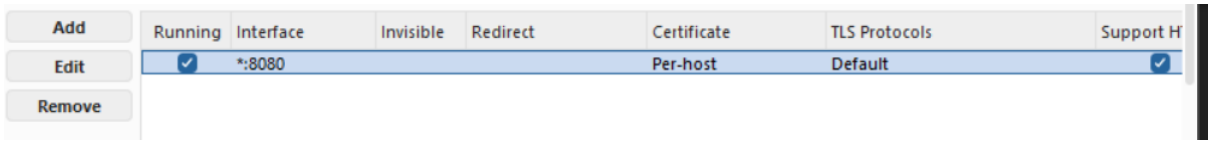


Figure 62 : shows burp listen on all interface on port 8080

I love burp to listen on all ports but you can utilize the way to like.

Now let configure our device proxy to the burp address and port.

Go to wifi settings, tap on setting and advanced the proxy settings, configure the burp address and port

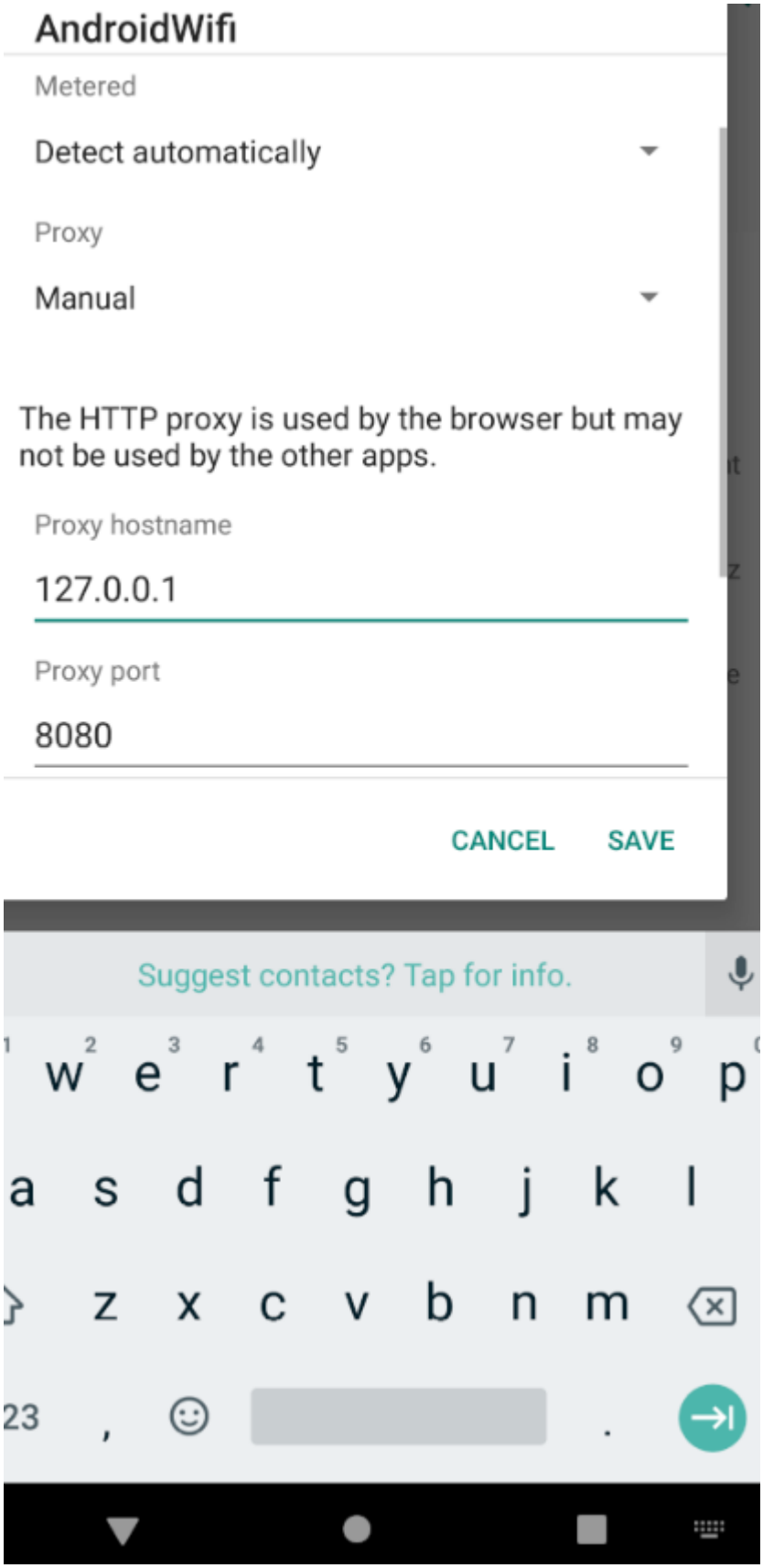


Figure 62 shows burp the configurations and then save

Back to burp tap the Proxy tab the intercept tab, turn on interception.

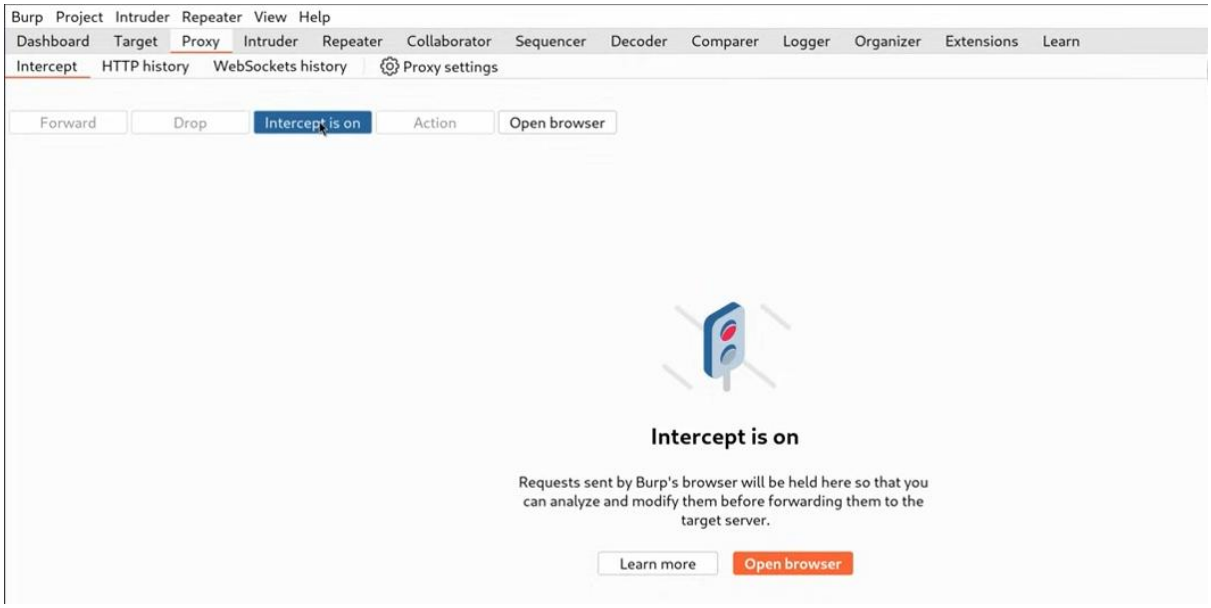


Figure 63: Interception turned on in burpsuite

Now in the device navigate to the browser and type any domain to confirm that burp is interception web traffic.



Figure 64 : Shows example.com on the browser

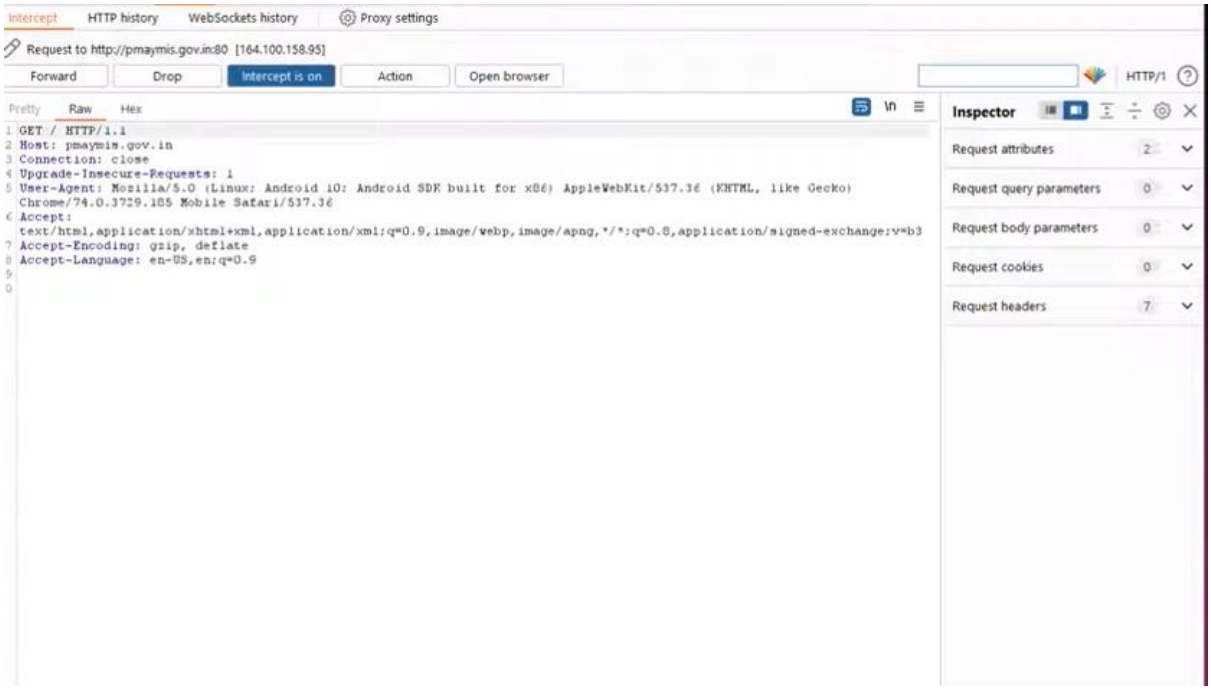


Figure 65: Shows that burp is intercepting traffic.

11.2 MANIPULATING BURP TRAFFIC

Now it is time that we use the Insecurebankv2 app to intercept its traffic.

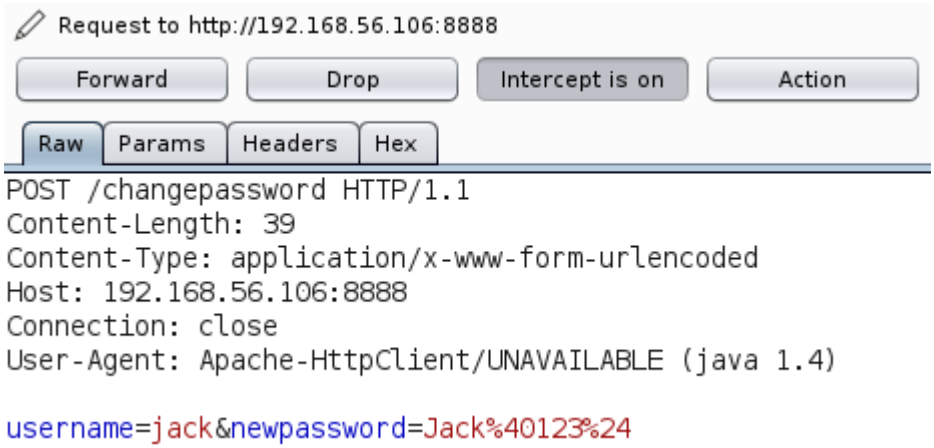


Figure 66 : shows the intercepted traffic when the user logs in, password and username is in plaintext.

Now I can manipulate the user input and see what will happen. Let say we go back to the app and click change password.

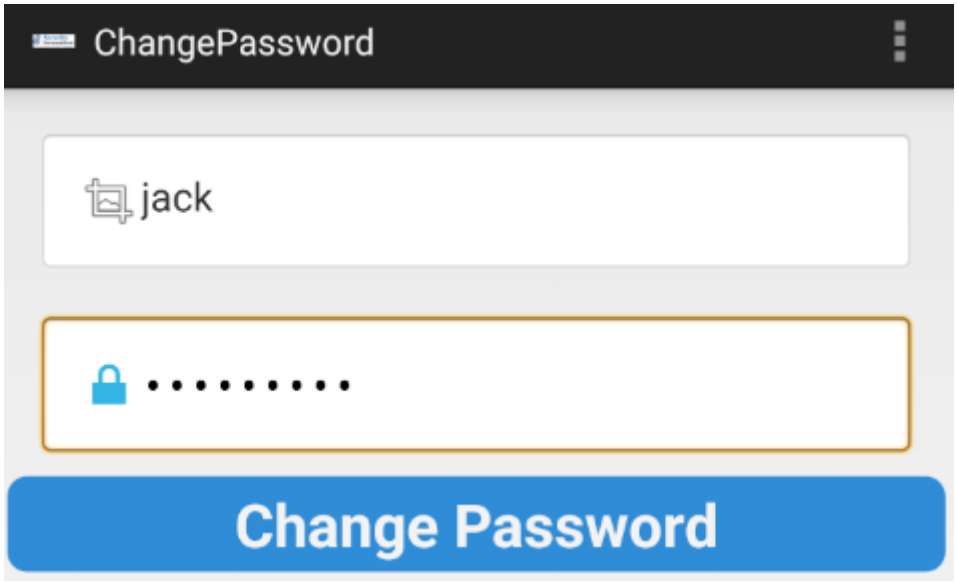


Figure 67: Shows change password with username jack and password.

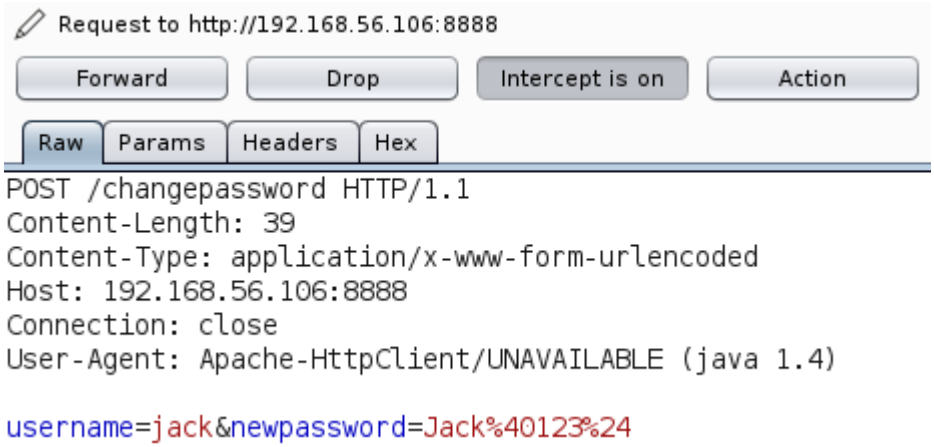


Figure 68: shows change password packet being cuptured with the user name and password

From there I can manipulate the packet and change its parameters to something else , for example I can change the username to maybe ”Thando”, dinesh , anything and change the password to another one.

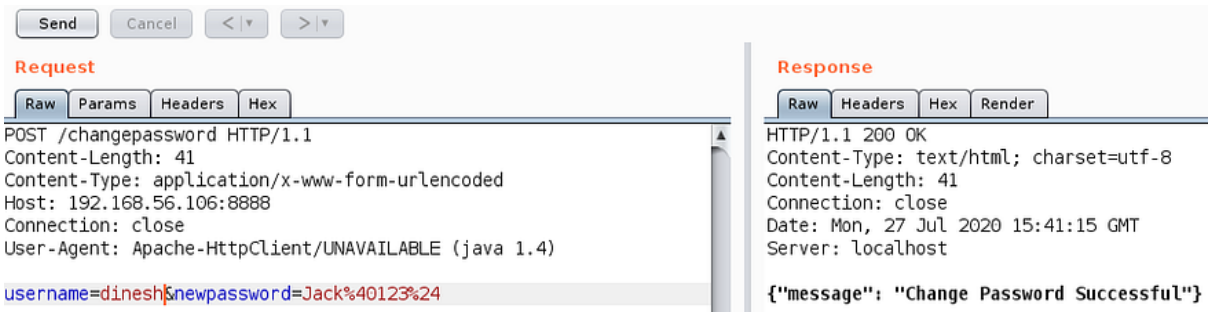


Figure 69: Shows manipulation of the parameters

Jack was changed to dinesh and now dinesh have a password of Jack4012324 , while this packets was to change jack password not dinesh password. Message shows ‘Change

password Successful’

12. FINDINGS AND ANALYSIS

Key Findings from Static Analysis

- Manifest inspection revealed dangerous permissions and exported components.
- Decompilation exposed potential hardcoded credentials, weak cryptographic implementations, and insecure WebView configurations.
- Source-code review (JD-GUI and JADX) allowed tracing of data flows and identification of insecure storage practices.

Key Findings from Dynamic Analysis (Drozer)

- The InsecureBankv2 application is highly vulnerable: multiple exported activities with null permissions, debuggable flag enabled, and exposed content providers.
- These issues allow an attacker to launch screens, inject data, or extract sensitive information without authentication.

Key Findings from Burp Suite Setup

- Successful certificate installation on a rooted device enables full man-in-the-middle (MITM) interception of HTTPS traffic, revealing API endpoints, tokens, and unencrypted data in transit.
- **Identifying the Flaw (Parameter Manipulation)** In the captured request, I noticed that the application sends the username as a visible parameter username along with the new password . This immediately flagged a potential security risk: the server might be relying on the parameter sent by the client rather than the active session to identify which user's password to update.
- **Executing the Attack** To test this, I modified the username parameter from "jack" to "dinesh" and forwarded the request to the server. To my surprise, the server responded with a success message. This confirmed a critical **Insecure Direct Object Reference (IDOR)** vulnerability.

Risk Analysis Exported components without proper permissions violate Android security best practices and enable confused deputy attacks. A debuggable flag in production apps allows memory dumping and runtime manipulation. Insecure IPC and lack of certificate pinning facilitate data exfiltration. Economically, such vulnerabilities can lead to data breaches, regulatory fines (e.g., GDPR/POPIA), and loss of user trust.

Recommendations

- Set android:exported="false" for non-essential components.
- Disable debuggable flag in release builds.
- Implement proper certificate pinning and input validation.

- Conduct regular static and dynamic scans during the SDLC.

13. Conclusion

In this project I have successfully delivered a comprehensive and meticulously documented exploration of Android application penetration testing, traversing the full spectrum from static inspection through to advanced dynamic exploitation. The journey began with the fundamental yet essential steps of **APK acquisition and extraction**, where I unpacked and examined the structural composition of Android application packages. Through the application of APKTool, the binary contents of the InsecureBankv2 application were transformed into accessible, human-readable formats, revealing the underlying Smali bytecode, XML manifest files, and resource directories. This initial phase established a critical foundation for all subsequent analysis, demonstrating that even superficial static inspection can uncover telling indicators of insecure development practices, such as overly broad permission declarations, exposed application components, and verbose debugging artefacts inadvertently left within production builds.

Through the deployment of **Android Debug Bridge (ADB)** I can certainly prove that it was an invaluable capability throughout the project, serving not merely as a utility for installing applications but as a powerful instrument for deep device interaction and data extraction. The commands that I used such as *adb devices*, *adb install*, *adb push* and *adb pull*, *adb backup*, and *adb shell dumpsys* were each exercised and documented, illustrating the breadth of control that ADB affords to security assessors. These commands enabled me to do the extraction of application data, inspection of running processes and services, modification of display parameters, and enumeration of installed packages , all of which are routine yet vital tasks during a professional mobile security engagement. The hands-on experience I gained through ADB interaction reinforces the understanding that Android device administration and forensic data collection are core competencies for any practitioner in this domain.

The **advanced decompilation** phase substantially deepened my analytical rigour of this project. By leveraging Dex2jar in conjunction with JD-GUI, the compiled DEX bytecode of multiple APK files was reverse-engineered into approximate Java source code, enabling me to read a line-by-line review of application logic. I also introduced JADX as an alternative decompilation environment which further enriched the process, offering me a more streamlined and visually intuitive interface for navigating complex codebases. These techniques collectively illuminated numerous security-relevant findings, including the presence of hardcoded API keys and credentials, weak or obsolete cryptographic algorithms, insecure WebView configurations susceptible to JavaScript injection, and poorly validated input handling routines. Moreover, my ability to trace data flows from user input through to storage or network transmission proved particularly revealing, underscoring how static source-code analysis can proactively identify vulnerabilities before an application is ever executed in a live environment.

The **dynamic analysis** components of my project, that I conducted using Drozer and Burp Suite, transitioned the investigation from theoretical code review to practical runtime exploitation. Drozer's systematic enumeration of the InsecureBankv2 application's attack surface exposed five exported activities, one exported broadcast receiver, one exported content provider, and critically, the presence of the *android:debuggable="true"* flag — a

configuration that should never exist in a production release. These findings represent severe deviations from Android security best practices and collectively create multiple pathways for unauthorised access, data exfiltration, and privilege escalation. The exploitation of exported activities without permission requirements demonstrated how an attacker could bypass authentication screens and directly invoke sensitive functionality. Similarly, the probing of content providers revealed potential vectors for SQL injection and structured data manipulation.

My configuration and operationalisation of **Burp Suite** as an intercepting HTTPS proxy represented a pivotal milestone my dynamic analysis workflow. The meticulous process of exporting the Burp Certificate Authority (CA) certificate, converting it into the Android-compatible format, and installing it within the system trust store of a rooted emulator provided me an invaluable practical experience in man-in-the-middle (MITM) proxy setup. Once operational, Burp Suite successfully intercepted all network traffic generated by the InsecureBankv2 application, laying bare the plaintext transmission of usernames and passwords, the absence of certificate pinning, and most alarmingly, an **Insecure Direct Object Reference (IDOR)** vulnerability within the password change functionality. By successfully manipulation of the username parameter in a password change request , altering "jack" to "dinesh" and receiving a success confirmation , I can boldly say I have successfully demonstrated a critical server-side authorisation flaw that could enable account takeover attacks with devastating consequences for affected users.

Taken together, My findings from this project paint a sobering picture of the security vulnerabilities that can pervade Android applications when secure development principles are neglected. Exported components without appropriate permission guards violate the principle of least privilege and enable confused deputy attacks. The debuggable flag in release builds opens the door to runtime memory analysis, code injection, and debugger attachment by adversaries. Inadequate transport layer security and the absence of certificate pinning render applications susceptible to network eavesdropping and session hijacking. The IDOR vulnerability exemplifies a broader class of server-side logic flaws that cannot be remediated through client-side measures alone, emphasising the necessity of holistic security architecture that spans both mobile frontend and backend infrastructure.

Beyond the immediate technical findings, I have tried in this project to yield significant educational and professional value. The step-by-step documentation, enriched with screenshots, command outputs, and explanatory commentary, produces a reproducible reference guide suitable for academic assessment, professional portfolio presentation, and peer knowledge sharing. The integration of multiple specialised tools into a coherent testing methodology mirrors the workflows employed by professional mobile security consultants and red-team operators in commercial environments. Furthermore, the ethical framework which I conducted all my testing , using deliberately vulnerable applications in isolated laboratory settings ,I reinforced the professional responsibility that accompanies offensive security skills.

Looking forward, the insights I gained from this project strongly advocate me for the institutionalisation of mobile security testing within organisational software development lifecycles. I can urge that developers should be trained to recognise and avoid anti-patterns such as hardcoded secrets, exported components, and debug configurations in production

builds. Automated static analysis tools should be integrated into continuous integration and continuous deployment pipelines to catch vulnerabilities at the earliest possible stage. Dynamic and interactive application security testing should complement manual penetration testing to achieve comprehensive coverage. Organisations must also invest in threat modelling exercises specifically tailored to mobile application architectures, accounting for risks arising from device loss, malicious applications, network adversaries, and compromised backend services.

In closing, my project stands as a testament to the reality that mobile security is not an afterthought to be appended at the conclusion of development, but rather a continuous, integral discipline that must be woven into every phase of design, implementation, testing, and deployment. The Android ecosystem, for all its innovation and accessibility, demands vigilance, expertise, and an unwavering commitment to protecting the privacy and security of end-users. Through the rigorous application of static and dynamic analysis techniques, my report has demonstrated both the prevalence of mobile vulnerabilities and the practical capacity to detect, understand, and ultimately remediate them. It is my sincere hope that this work will serve as a valuable learning resource and inspire continued pursuit of excellence in the ever-evolving field of mobile application security.

14. References

1.]Makan, K. (2016) *Learning Android Penetration Testing*. Birmingham: Packt Publishing.
2. **[5]**Srinivasan, R. (2019) *Hands-On Mobile App Security*. Chennai: Notion Press.
3. **[8]**OWASP Foundation (2024) *OWASP Mobile Security Testing Guide (MSTG)*. Available at: https://mas.owasp.org/ (Accessed: 15 May 2026).
4. **[9]**OWASP Foundation (2024) *OWASP Mobile Application Security Verification Standard (MASVS)*. Available at: https://mas.owasp.org/MASVS/ (Accessed: 15 May 2026).
5. **[11]**Android Open Source Project (2024) *Android Debug Bridge (ADB) Documentation*. Available at: https://developer.android.com/studio/command-line/adb (Accessed: 10 May 2026).
6. **[12]**Android Open Source Project (2024) *APK File Format and Structure*. Available at: https://source.android.com/docs/core/architecture/android-apk (Accessed: 10 May 2026).
7. **[13]**Android Open Source Project (2024) *Application Security — Android Developers Guide*. Available at: https://developer.android.com/privacy-and-security (Accessed: 10 May 2026).
8. **[14]**Google LLC (2024) *Google Play Protect — Security Overview*. Available at: https://developers.google.com/android/play-protect (Accessed: 12 May 2026).
9. **[15]**PortSwigger Ltd (2024) *Burp Suite Professional Documentation*. Available at: https://portswigger.net/burp/documentation (Accessed: 14 May 2026).
10. **[16]**PortSwigger Ltd (2024) *Web Security Academy — Mobile Application Security*. Available at: https://portswigger.net/web-security/mobile (Accessed: 14 May 2026).
11. **[17]**WithSecure Labs (2024) *Drozer User Guide and Framework Documentation*. Available at: https://labs.withsecure.com/tools/drozer/ (Accessed: 13 May 2026).
12. Shetty, D. (2017) *Android-InsecureBankv2: An Intentionally Vulnerable Android Application for Security Professionals and Students*. GitHub repository. Available at: https://github.com/dineshshetty/Android-InsecureBankv2 (Accessed: 12 May 2026).